

HTML5 JS API

講師：林新德 shinder.lin@gmail.com

講義版本：2026-09-09

範例專案：<https://github.com/shinder/html5-js-api-examples>

目錄

0. Web APIs	p. 4
0.1 將介紹的主題	p. 4
0.2 W3C 規範流程與工具	p. 4
0.3 環境準備	p. 5
1. 新增的標籤與屬性	p. 6
1.1 Semantic Elements	p. 7
1.2 表單的 input	p. 7
1.3 Contenteditable	p. 12
2. 嵌入多媒體：Audio 與 Video	p. 12
2.1 Audio	p. 12
2.2 Video	p. 14
3. Canvas	p. 16
3.1 畫線與多邊形	p. 16
3.2 矩形	p. 18
3.3 圓、弧、扇形	p. 19
3.4 畫圖片	p. 20
3.5 畫文字	p. 21
3.6 截取影片	p. 22
3.7 塗鴉練習	p. 22
3.8 動畫	p. 24
4. SVG	p. 25
4.1 常見的 SVG 標籤	p. 26
4.2 範例：互動式 SVG	p. 26
5. Drag and Drop (拖放)	p. 27
5.1 讓元素能被拖	p. 28
5.2 拖到目標再放開	p. 28
5.3 用 dataTransfer 傳資料	p. 29
5.4 多個拖曳元素	p. 29
5.5 手刻拖放 (選讀)	p. 30
6. FileList、File、FileReader	p. 30
6.1 取得 File 物件	p. 31

6.2 FileReader：讀取檔案內容	p. 32
7. Web Storage	p. 33
7.1 共用方法	p. 34
7.2 使用 localStorage	p. 34
7.3 使用 sessionStorage	p. 38
8. Server-Sent Events (SSE)	p. 39
8.1 伺服器送出格式	p. 39
8.2 伺服器：每兩秒推一次時間	p. 39
8.3 用戶端：用 EventSource 接收	p. 40
9. WebSocket	p. 41
9.1 WebSocket 用戶端 API	p. 41
9.2 ws：Node.js WebSocket library	p. 42
9.3 設定：三條 WebSocket 共用同一個 port	p. 42
9.4 回音程式	p. 43
9.5 Map 和 Set 型別	p. 44
9.6 簡易聊天室	p. 45
9.7 共用塗鴉板	p. 46
10. AJAX 上傳檔案	p. 48
10.1 後端：multer 設定模組	p. 48
10.2 單檔上傳：個人資料 + 大頭貼	p. 49
10.3 多檔上傳：相簿	p. 51
11. Web Worker	p. 52
11.1 Dedicated Worker	p. 52
11.2 Shared Worker	p. 54
12. PWA (Progressive Web App)	p. 55
12.1 PWA 解決什麼問題	p. 56
12.2 各平台支援情況	p. 56
12.3 三個關鍵組件	p. 56
12.4 Web App Manifest (宣告 App 身分)	p. 57
12.5 註冊 Service Worker	p. 58
12.6 離線快取	p. 59
12.7 自訂安裝按鈕	p. 60
附錄一：Geolocation	p. 61
A.1 取得目前座標	p. 62
A.2 座標搭配地圖	p. 62
附錄二：用 OpenAI SDK 串接本機 Ollama	p. 64
B.1 環境準備	p. 64
B.2 兩種架構	p. 64
B.3 前端直接呼叫 Ollama	p. 64

B.4 透過後端中繼（保護 API key）	p. 66
附錄三：用 Cloudflare Tunnel 在本機開 HTTPS 測試環境	p. 68
C.1 安裝 cloudflared	p. 68
C.2 啟動 quick tunnel	p. 69
C.3 典型應用情境	p. 69
C.4 注意事項與進階用法	p. 69

0. Web APIs

HTML5 JS API 是瀏覽器內建的 JavaScript 介面集合，讓網頁能做以前必須靠外掛（Flash、Silverlight）或原生 App 才能做的事 — 繪圖、播放多媒體、操作檔案、與伺服器即時雙向通訊、利用多執行緒、取得地理位置、安裝為桌面 App... 通通內建在瀏覽器，不必額外安裝。



圖 1 HTML5 主要功能分類

- 主題列表（MDN）：<https://developer.mozilla.org/en-US/docs/Web/API>
- 各功能瀏覽器支援度：<https://caniuse.com/>

0.1 將介紹的主題

編號	主題
§1	新增的標籤與屬性（語意化、Form input、Contenteditable）
§2	嵌入多媒體：Audio 與 Video
§3	Canvas（2D 繪圖、影像、動畫）
§4	SVG
§5	Drag and Drop
§6	FileList、File、FileReader
§7	Web Storage
§8	Server-Sent Events（SSE）
§9	WebSocket
§10	AJAX 上傳檔案
§11	Web Worker
§12	PWA（Progressive Web App）

附錄：

- 附錄一：Geolocation
- 附錄二：用 OpenAI SDK 串接本機 Ollama
- 附錄三：用 Cloudflare Tunnel 在本機開 HTTPS 測試環境

0.2 W3C 規範流程與工具

W3C 規範的成熟度由低到高：

縮寫	全名	中文
WD	Working Draft	工作草案
CR	Candidate Recommendation	候選推薦標準
PR	Proposed Recommendation	提案推薦標準
REC	W3C Recommendation	W3C 推薦標準

詳細：[https://zh.wikipedia.org/wiki/W3C 推薦標準](https://zh.wikipedia.org/wiki/W3C_推薦標準)

HTML 本身的現況：2019 年起 W3C 與 WHATWG 達成協議，HTML 與 DOM 規範改由 WHATWG 以「Living Standard」持續維護 (<https://html.spec.whatwg.org/>)，不再發行 HTML5、HTML5.1 這類版本號。上面的流程主要適用於 CSS、SVG、Web API 等仍在 W3C 的規範。

常用查證工具：

- caniuse.com：查某 API 在各瀏覽器的支援度
- Chrome DevTools → Application：檢視 PWA Manifest、Service Worker、Storage
- Chrome DevTools → Network：模擬離線、限速、看 SSE/WebSocket 訊息

0.3 環境準備

§8、§9、§10、附錄二的範例需要 Node.js 後端 — 用 Express 起一個簡單的 server。其他單元 (§1—§7、§11、§12、附錄一) 只需要 server 把靜態檔送出去即可，套件不必裝齊。

嫌麻煩可直接 clone 範例專案：<https://github.com/shinder/html5-js-api-examples>，
`npm i` 後 `npm run dev` 即可。

範例專案另附 `csharp-backend/`，是同一組後端功能 (§8、§9、§10) 的 ASP.NET Core 版本，前端 HTML 共用；說明見講義的 .NET 補充版 `html5-api-dotnet.md`。

1. 建立專案資料夾並切換進去：

```
mkdir html5-js-api-base
cd html5-js-api-base
```

2. 初始化 npm 專案：

```
npm init -y
```

3. 安裝核心套件：

```
npm i express serve-index
npm i --save-dev nodemon
```

4. 修改 `package.json`，加入 `type` 與 `scripts`：

```
{
  "type": "module",
  "scripts": {
    "dev": "nodemon index.js"
  }
}
```

5. 建立 `public/` 資料夾與 `index.js` :

```
mkdir public
```

`index.js` :

```
import express from "express";
import serveIndex from "serve-index";

const web_port = process.env.WEB_PORT || 3031;
const app = express();

app.use(express.urlencoded({ extended: true }));
app.use(express.json());
app.use(express.static("public"));
app.use("/", serveIndex("public", { icons: true }));

app.listen(web_port, () => {
  console.log(`伺服器啟動於通訊埠: http://localhost:${web_port}`);
});
```

6. 啟動 :

```
npm run dev
```

開啟 <http://localhost:3031>，會看到 `public/` 的檔案列表。

後續視需要再裝的套件

套件	用途	出現章節
<code>ws</code>	WebSocket server	§9
<code>multer</code>	上傳檔案的 multipart 解析	§10
<code>uuid</code>	產生隨機檔名	§10
<code>openai</code>	OpenAI SDK (也用來連 Ollama)	附錄二

1. 新增的標籤與屬性

HTML5 在標籤層級補上了三件以前要靠 `div` + `class` 或 JS 才能完成的事：用語意化標籤描述頁面結構、讓 `input` 接管常見的輸入型別與驗證、以及把任何元素變成可編輯。下面分三小節介紹。

1.1 Semantic Elements

在 HTML5 之前，整個頁面常常是一堆 `<div class="header">`、`<div class="sidebar">`，搜尋引擎與螢幕報讀軟體無法判斷哪一塊是「主要內容」、哪一塊是「導覽列」。語意化標籤把這些角色直接寫在標籤名稱上，結構自帶意義，外觀則交給 CSS 處理。

標籤	用途
<code><header></code>	頁首，可放 logo、標題
<code><nav></code>	導覽列
<code><main></code>	頁面唯一的主要內容
<code><section></code>	一段獨立主題的區塊
<code><article></code>	可獨立散布的內容（部落格文章、新聞）
<code><aside></code>	與主內容相關但可分離的內容（側欄）
<code><footer></code>	頁尾或區塊尾
<code><figure></code> / <code><figcaption></code>	含說明文字的插圖、程式碼或影片
<code><details></code> / <code><summary></code>	可摺疊的內容區塊（瀏覽器原生實作）
<code><mark></code>	標示關鍵字（黃色螢光標示）
<code><time></code>	機器可讀的日期/時間

參考：https://www.w3schools.com/html/html5_semantic_elements.asp

這些標籤在外觀上和 `<div>` 完全一樣，差別在「機器」能讀懂結構。一個典型的版面配置：

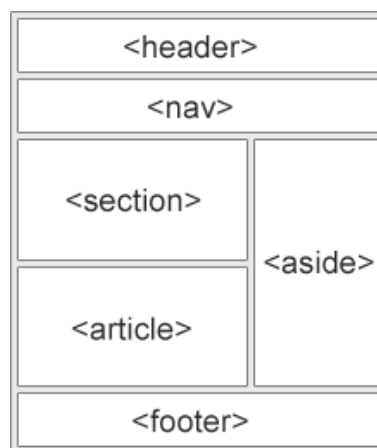


圖 2 Semantic Elements 版面配置

1.2 表單的 input

`input` 在 HTML5 加上一系列新的 `type`，最直接的好處是行動瀏覽器會依型別切換鍵盤（`tel` 跳數字鍵盤、`email` 鍵盤多 `@` 鍵、`url` 鍵盤多 `.com`），桌面瀏覽器則自動冒出原生的選色器、日期選擇器，省下大量 JS 與第三方套件。

範例：`101-form-input-type.html`

```
<form onsubmit="return false;">
  <label for="myColor">顏色</label>
  <input type="color" id="myColor" /><br />

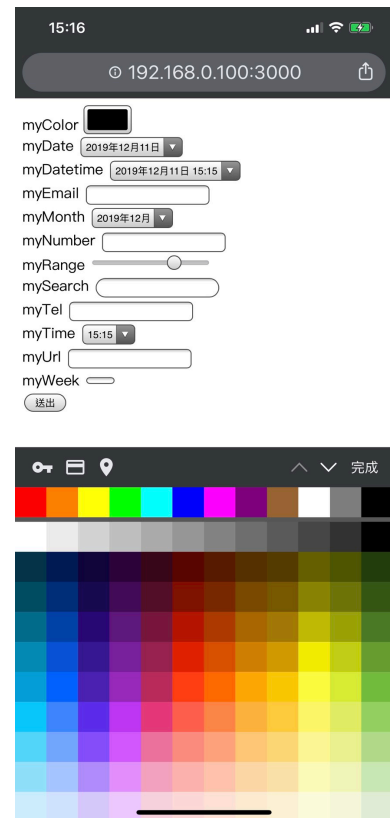
  <label for="myDate">日期</label>
  <input type="date" id="myDate" /><br />

  <label for="myEmail">電郵</label>
  <input type="email" id="myEmail" /><br />

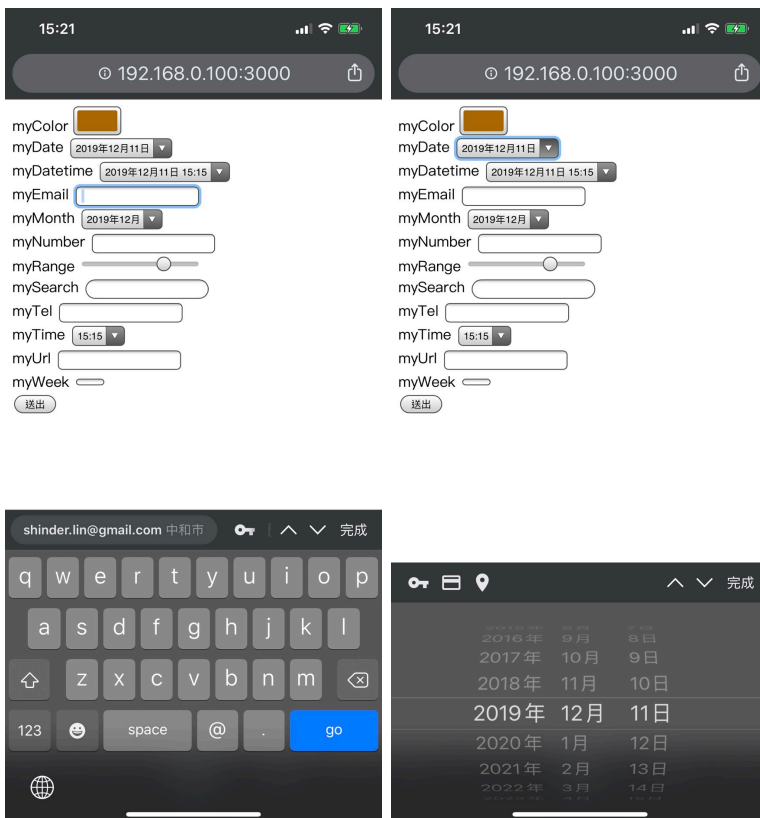
  <label for="myRange">滑桿</label>
  <input type="range" id="myRange" min="5" max="15" step="2" /><br />

  <label for="myTel">電話</label>
  <input type="tel" id="myTel" /><br />

  <!-- 還有 datetime-local, month, number, search, time, url, week -->
  <input type="submit" />
</form>
```



行動瀏覽器（iOS Safari）對部分型別會跳出對應的原生介面：



新的屬性

`input` 與 `form` 也加了幾個常用屬性，搭配 `type` 就能不寫 JS 完成基本驗證：

屬性	用途
<code>required</code>	必填
<code>pattern</code>	用 regexp 限定格式
<code>placeholder</code>	佔位提示文字
<code>autofocus</code>	頁面載入後自動聚焦
<code>autocomplete</code>	開啟瀏覽器記憶 (on / off)
<code>min / max / step</code>	數值與日期範圍
<code>multiple</code>	可選多個檔案或多個 email
<code>list</code>	搭配 <code><datalist></code> 提供建議選項

參考：https://www.w3schools.com/html/html_form_attributes.asp

範例：`102-form-input-pattern.html` 示範 `required`、`autofocus`、`pattern`、`<datalist>` 的綜合用法：

```
<form onsubmit="return false;" autocomplete="on">
  <label for="myName">姓名：</label>
  <input type="text" id="myName" required autofocus /><br />

  <label for="myEmail">電郵：</label>
  <input type="email" id="myEmail" /><br />

  <label for="myGroup">組別：</label>
  <input type="text" id="myGroup" list="myList" />
  <datalist id="myList">
    <option value="甲組"></option>
    <option value="乙組"></option>
    <option value="丙組"></option>
  </datalist><br />

  <label for="myTel">手機號碼：</label>
  <input type="tel" id="myTel" pattern="09\d{2}-?\d{3}-?\d{3}" /><br />

  <input type="submit" />
</form>
```

`<datalist>` 提供「建議選項」但不限制使用者一定要從中選，比 `<select>` 自由；`pattern` 失敗時瀏覽器會自動跳泡阻擋送出。

`<datalist>` 注意事項

1. 下拉清單的樣式（字型、寬度、配色）無法用 CSS 客製，由瀏覽器決定。需要美化的 `autocomplete` 必須自己用 `div` + 鍵盤事件 + `aria-*` 重做。

2. 各瀏覽器的觸發時機不一致：Chrome / Safari 要開始輸入才跳建議，Firefox — focus 就跳全部。
3. 與 `type="color"`、`type="date"`、`type="time"`、`type="month"` 搭配的支援度零散，跨瀏覽器要一致建議自己處理。`type="text"` 則無此問題。

自訂驗證訊息

`required` / `pattern` 失敗時，瀏覽器顯示的預設訊息（如「請填妥這個欄位」）通常不夠貼近情境。用 `setCustomValidity()` 可以換成自訂文字 — 只要訊息字串不是空字串，欄位就被視為無效；通過時要記得傳入空字串清掉訊息。

範例：103-form-custom-validity.html — 確認密碼欄與密碼欄一致：

```
const myPass = document.querySelector("#myPass");
const myPass2 = document.querySelector("#myPass2");

myPass2.addEventListener("blur", () => {
  if (myPass.value !== myPass2.value) {
    myPass2.setCustomValidity("密碼確認欄必須和密碼欄相同內容");
  } else {
    myPass2.setCustomValidity(""); // 空字串 = 通過
  }
});
```

進一步可讀取 `input.validity` 物件，依失敗原因（缺值、格式錯、型別錯...）給不同訊息。範例：104-form-custom-validity-2.html：

```
const myValidate = (event) => {
  const t = event.target;
  const v = t.validity; // ValidityState
  let msg = "";
  switch (true) {
    case v.valueMissing: msg = "必填欄位"; break;
    case v.typeMismatch: msg = "請填寫正確的類型"; break;
    case v.patternMismatch: msg = "請填寫規定的格式"; break;
  }
  t.setCustomValidity(msg);
};

document.querySelectorAll("input").forEach((field) => {
  field.addEventListener("blur", myValidate);
});
```

`ValidityState` 還有 `rangeOverflow`、`rangeUnderflow`、`stepMismatch`、`tooLong`、`tooShort`、`badInput`（如 `type="number"` 填了非數字）、`customError`（目前有 `setCustomValidity` 設下的訊息）等屬性，可依需要組合判斷。

1.3 Contenteditable

在任何元素上加 `contenteditable="true"`，使用者就能直接點進去編輯內容；搭配 `input` 事件可即時取得目前內容。常用來做行內編輯（inline editing）或簡易的 WYSIWYG 編輯器，不必再額外做一個 `textarea` + 按鈕的編輯模式。

範例：`105-contenteditable.html`

```
<style>
  #info { background-color: #00b7ff; }
  #info:focus { background-color: #ffc44e; }
</style>

<div
  id="info"
  contenteditable="true"
  oninput="console.log(this.innerText);"
>
  123
</div>
```

取「純文字」用 `innerText`，取「含格式 HTML」用 `innerHTML`；前者較安全，後者要留意 XSS。

2. 嵌入多媒體：Audio 與 Video

HTML5 之前在頁面上播放聲音或影片必須靠 Flash 或第三方外掛。HTML5 直接把這件事內建成

`<audio>` 與 `<video>` 兩個標籤：瀏覽器會自帶播放介面（控制條、音量、全螢幕），同時提供完整的 JS 操作 API，可以監聽事件、動態切換來源、做截圖、加字幕。

兩個標籤的型別關係：

- `<audio>` → `HTMLAudioElement`
- `<video>` → `HTMLVideoElement`
- 兩者都繼承自 `HTMLMediaElement`，共通的事件（`loadedmetadata`、`timeupdate`、`ended`、`play`、`pause`）與屬性（`duration`、`currentTime`、`volume`、`muted`、`paused`）都在父類別定義。

2.1 Audio

`<audio>` 用來在頁面上嵌入聲音檔。常見屬性：

屬性	說明
src	音訊檔路徑（也可改用內部 <code><source></code> 提供多格式 fallback）
controls	顯示瀏覽器內建的播放控制條
preload	auto / metadata / none，控制是否預載
loop	循環播放
autoplay	自動播放（目前主流瀏覽器須 muted 才能自動播放）

範例：201-audio-tag.html

```
<audio
  src="media/panflute-loop.mp3"
  controls
  preload="auto"
  loop
>
  <a href="media/panflute-loop.mp3">瀏覽器不支援時，提供下載連結</a>
</audio>
```

標籤內的內容（這裡的 `<a>`）只會在瀏覽器不支援 `<audio>` 時顯示，是 fallback 機制。

用 Audio 物件動態載入

如果聲音不需要顯示控制條（例如按鈕的提示音、遊戲音效），可以直接用 `new Audio()` 建立物件，不必在 HTML 寫 `<audio>` 標籤。設定 `src` 後瀏覽器才會開始載入。

範例：202-audio-object.html

```
<button id="doLoad">載入</button>
<button id="doPlay" style="display: none">播放或停止</button>
```

```
const au = new Audio();

au.addEventListener("loadedmetadata", () => {
  console.log("已載入完成，總長度：", au.duration, "秒");
});
au.addEventListener("timeupdate", () => {
  console.log("目前播到：", au.currentTime);
});
au.addEventListener("ended", () => {
  console.log("播完了");
});

doLoad.onclick = () => {
  au.src = "media/panflute-loop.mp3"; // 設定 src 後才開始載入
  doLoad.style.display = "none";
  doPlay.style.display = "block";
};
doPlay.onclick = () => (au.paused ? au.play() : au.pause());
```

範例裡直接用 `doLoad`、`doPlay` 當變數，是因為瀏覽器會把有 `id` 的元素自動掛成全域變數。這是方便示範的寫法，正式專案仍建議用 `querySelector` 明確取得元素。後面多個範例也採同樣寫法。

`HTMLMediaElement` 常用事件：

- `loadedmetadata`：metadata（時長、尺寸）載入完成
- `timeupdate`：播放位置變動（每幾百毫秒觸發一次）
- `play` / `pause`：開始播放 / 暫停
- `ended`：播放到結尾

2.2 Video

`<video>` 多了視覺面的屬性（`width`、`height`、`poster`），以及無聲自動播的 `muted` 慣用搭配。

屬性	說明
<code>poster</code>	未播放前顯示的封面圖
<code>width</code> / <code>height</code>	顯示尺寸
<code>muted</code>	靜音；搭配 <code>autoplay</code> 才能繞過自動播放限制
<code>controls</code> , <code>preload</code> , <code>loop</code> , <code>autoplay</code> , <code>src</code>	與 <code><audio></code> 相同

自動播放限制：主流瀏覽器（Chrome / Safari / Firefox）都禁止「有聲」自動播放，必須加 `muted`，或等使用者觸發過互動之後。

範例：`203-video-tag.html`

```
<video
  src="media/big-buck-bunny_trailer.webm"
  poster="images/bunny-poster.png"
  width="320"
  height="240"
  controls
  loop
  autoplay
  muted
>
  <p>瀏覽器不支援時顯示這段文字</p>
</video>
```

Demo 影片來源：<https://www.webmfiles.org/demo-files/>

動態切換影片來源

直接改 `video.src` 就會載入新檔案；事件監聽器掛一次，每次切換都會生效。下面用 `data-src` 屬性把影片路徑寫在按鈕上，比 `switch` 比對 `id` 容易擴充。

範例：`204-video-object.html`

```
<video id="player" controls></video>
<div id="btnGroup">
  <button data-src="media/big-buck-bunny_trailer.webm">影片 1</button>
  <button data-src="media/elephants-dream.webm">影片 2</button>
  <button id="doPlay">播放/暫停</button>
</div>
```

```
const player = document.querySelector("#player");

player.addEventListener("loadedmetadata", () => {
  console.log("新影片載入，時長：", player.duration);
});

document.querySelector("#btnGroup").addEventListener("click", (e) => {
  const src = e.target.dataset.src;
  if (src) {
    player.src = src;          // 切換來源 → 自動觸發 loadedmetadata
  } else if (e.target.id === "doPlay") {
    player.paused ? player.play() : player.pause();
  }
});
```

字幕：WebVTT

`<track>` 標籤可以掛字幕檔（`.vtt`），瀏覽器自動顯示字幕切換選單。`default` 屬性指定預設啟用的字幕軌；多個 `<track>` 可以同時掛不同語系。

範例：`205-video-subtitles.html`

```
<video
  src="media/big-buck-bunny_trailer.webm"
  poster="images/bunny-poster.png"
  preload="auto"
  controls
>
  <track kind="subtitles" label="正體中文" src="media/bunny-zh.vtt" default />
  <track kind="subtitles" label="English" src="media/bunny-en.vtt" />
</video>
```

WebVTT 檔的格式（`media/bunny-zh.vtt`）：

WEBVTT

NOTE 這是 VTT 註解

```
1
00:00:05.000 --> 00:00:08.000
影片剛開始

2
00:00:12.000 --> 00:00:20.000
這是第二句

3
00:00:20.000 --> 00:00:25.000
第三
```

1. 第一行 `WEBVTT` 之後**必須空一行**才能接 `NOTE` 或字幕段落，否則會被當成標頭的一部分。
2. 字幕檔必須是 UTF-8 編碼，與 HTML 同源（跨網域要設 CORS）。
3. 詳細格式參考：https://developer.mozilla.org/zh-TW/docs/Web/API/Web_Video_Text_Tracks_Format

3. Canvas

`<canvas>` 是 HTML5 引入的點陣圖繪圖區域，所有繪圖都透過 JS 呼叫 2D context (`getContext("2d")`) 的方法完成。它是「立即模式」的 API — 一畫上去就是像素，瀏覽器不再追蹤個別物件，所以效能好、適合做動畫、遊戲、影像處理；但要操作個別圖形得自己維護資料結構。

Canvas vs SVG：SVG 是向量、保留物件樹 (DOM)，可個別操作圖形、容易處理事件；Canvas 是點陣、無物件樹，適合大量物件或像素級操作（粒子、濾鏡、遊戲）。

常用 Canvas 函式庫：[pixi.js](#) (2D 遊戲)、[three.js](#) (3D，用 WebGL context)、[fabric.js](#) (在 Canvas 上加物件層，補回個別操作能力)。

3.1 畫線與多邊形

Canvas 用「路徑 (path)」的概念畫線：先 `beginPath()` 開啟新路徑，用 `moveTo()` 移動畫筆、`lineTo()` 連到下一個點，最後 `stroke()` 沿路徑畫線 (`fill()` 則填滿)。

範例：`301-canvas-line-1.html`

```
<canvas id="myCanvas" width="600" height="400"></canvas>
<script>
  const myCanvas = document.querySelector("#myCanvas");
  const ctx = myCanvas.getContext("2d");

  ctx.beginPath();           // 開新路徑
  ctx.moveTo(50, 50);        // 起點
```

```

ctx.lineTo(200, 300);      // 連到第二點
ctx.lineTo(400, 50);      // 連到第三點
ctx.strokeStyle = "orange";
ctx.lineWidth = 20;
ctx.stroke();              // 沿路徑畫線
</script>

```

Canvas 預設大小是 300x150 px。繪圖解析度由 HTML 的 `width / height` 屬性決定；CSS 的 `width / height` 只改變顯示尺寸，等於把畫好的點陣圖拉伸，單獨使用會變形。要做響應式畫布，作法是 CSS 控制外觀尺寸、再用 JS 依 `devicePixelRatio` 換算後設定 HTML 屬性。

線條屬性

線條兩端的造型（`lineCap`）與轉角的造型（`lineJoin`）會大幅影響視覺：

屬性	可選值	說明
<code>lineCap</code>	<code>butt</code> （預設）、 <code>round</code> 、 <code>square</code>	線段端點造型
<code>lineJoin</code>	<code>miter</code> （預設）、 <code>round</code> 、 <code>bevel</code>	線段轉角的銜接造型
<code>closePath()</code>	—	將路徑尾接回起點，封閉路徑

範例：302-canvas-line-2.html（讓使用者切換值看效果）

```

<canvas id="myCanvas" width="600" height="400"></canvas>
<form name="form1">
  <input type="checkbox" name="isClosedPath" id="isClosedPath" />
  <label for="isClosedPath">封閉</label>
  <select name="lineCap">
    <option value="butt" selected>butt</option>
    <option value="round">round</option>
    <option value="square">square</option>
  </select>
  <select name="lineJoin">
    <option value="miter" selected>miter</option>
    <option value="round">round</option>
    <option value="bevel">bevel</option>
  </select>
</form>

```

```

const draw = () => {
  ctx.clearRect(0, 0, myCanvas.width, myCanvas.height);
  ctx.beginPath();
  ctx.moveTo(50, 50);
  ctx.lineTo(200, 300);
  ctx.lineTo(400, 50);
  if (document.form1.isClosedPath.checked) ctx.closePath();
  ctx.strokeStyle = "orange";
  ctx.lineWidth = 40;
}

```

```

ctx.lineCap = document.form1.lineCap.value;
ctx.lineJoin = document.form1.lineJoin.value;
ctx.stroke();
};

// form 內任何欄位變動都重畫
document.form1.addEventListener("change", draw);
draw();

```

`document.form1.lineCap` 這種寫法靠的是 `<form name="form1">` 與欄位的 `name` 屬性，是表單專用的捷徑，不需要 `querySelector`。



圖 3 `lineCap=round, lineJoin=round` 的效果

3.2 矩形

矩形有三個一行就能完成的快捷方法，不需要 `path`：

方法	用途
<code>strokeRect(x, y, w, h)</code>	畫框（線條）
<code>fillRect(x, y, w, h)</code>	畫實心矩形
<code>clearRect(x, y, w, h)</code>	清除指定範圍（畫布變透明）

範例：`303-canvas-rect.html`

```

ctx.strokeStyle = "orange";
ctx.lineWidth = 20;
ctx.lineJoin = "round";
ctx.strokeRect(50, 50, 100, 100);

ctx.fillStyle = "lightblue";

```

```

ctx.fillRect(50, 200, 100, 100);

// 陰影效果（也適用於後續其他繪圖呼叫）
ctx.shadowOffsetX = 10;
ctx.shadowOffsetY = 10;
ctx.shadowColor = "rgba(0,0,0,.5)";
ctx.shadowBlur = 4;
ctx.strokeRect(350, 50, 100, 100);
ctx.fillRect(350, 200, 100, 100);

```

`shadow*` 屬性設了之後會持續套用到後續所有繪圖呼叫。要關掉就把 `shadowColor` 設為透明（`"rgba(0,0,0,0)"`）。

3.3 圓、弧、扇形

`arc()` 一個方法處理圓、弧、扇形三種狀況；差別只在「畫到什麼角度停下」與「填滿與否」。

```

ctx.arc(x, y, radius, startAngle, endAngle, anticlockwise);
//      圓心x 圓心y 半徑  起始角度  結束角度  是否逆時針(預設 false)

```

角度的單位是「弧度 (radian)」， $360^\circ = 2\pi$ 。常用： $90^\circ = \text{Math.PI} / 2$ 、 $180^\circ = \text{Math.PI}$ 、 $360^\circ = \text{Math.PI} * 2$ 。

範例：`304-canvas-arc.html`

```

ctx.strokeStyle = "red";
ctx.lineWidth = 6;

// 完整圓 - 起點 0、終點 2π
ctx.beginPath();
ctx.arc(100, 100, 50, 0, Math.PI * 2);
ctx.stroke();

// 1/4 弧
ctx.beginPath();
ctx.arc(250, 100, 50, 0, Math.PI / 2);
ctx.stroke();

// 扇形 - 弧 + lineTo 圓心 + fill
ctx.beginPath();
ctx.arc(400, 100, 50, 0, Math.PI / 2);
ctx.lineTo(400, 100);
ctx.fillStyle = "lightblue";
ctx.fill();

```

3.4 畫圖片

`drawImage()` 有三種呼叫方式，分別處理「定點繪製」、「縮放」、「切割」：

```
// 1. 定點繪製
ctx.drawImage(image, x, y);

// 2. 縮放
ctx.drawImage(image, x, y, width, height);

// 3. 切割來源圖再貼到目標
ctx.drawImage(image,
  sx, sy, sWidth, sHeight,    // 來源圖 (source) 的範圍
  dx, dy, dWidth, dHeight    // 貼到 canvas (destination) 的位置與尺寸
);
```

第三種形式是做「精靈圖 (sprite sheet)」的關鍵 — 在大圖中切出一格貼到 canvas，遊戲角色動畫常見的作法。

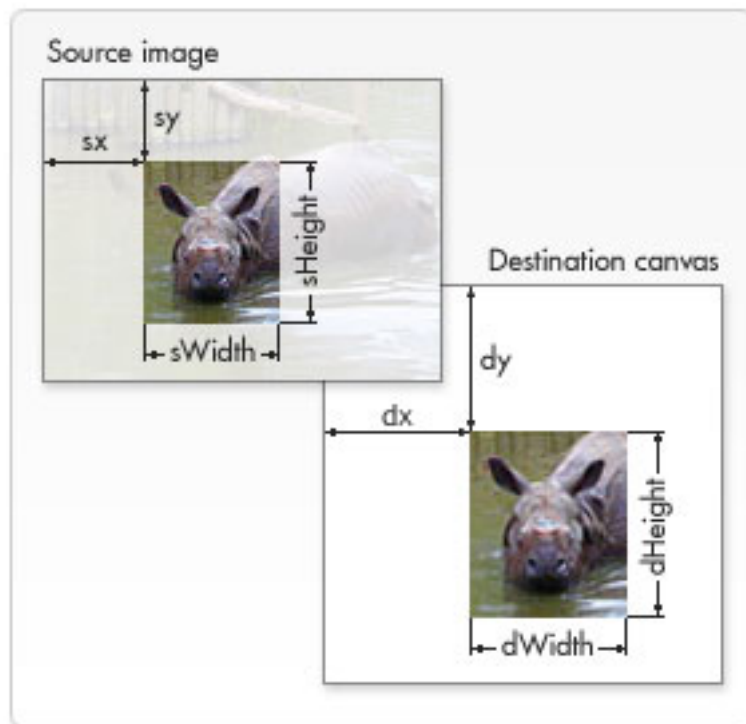


圖 4 `drawImage` 切割來源圖到目標 canvas

參考：https://developer.mozilla.org/zh-TW/docs/Web/API/Canvas_API/Tutorial/Using_images

等比例縮放

`Image` 物件載入是非同步的，必須在 `load` 事件後才能 `drawImage`，否則會畫不出來（圖根本還沒到）。

範例：305-canvas-image-1.html

```
const img = new Image();
img.addEventListener("load", () => {
  // 寬度填滿畫布、高度依比例縮放
  const w = myCanvas.width;
  const h = (img.height * w) / img.width;
  ctx.drawImage(img, 0, 0, w, h);
});
img.src = "images/bunny-poster.png";
```

從用戶端圖檔繪入

把 `<input type="file">` 選到的檔案丟給 `URL.createObjectURL()` 取得暫存 URL，再餵給 `Image` 即可。整個流程在前端完成，不用上傳到伺服器。

範例：306-canvas-image-2.html

```
<canvas id="myCanvas" width="600" height="400"></canvas>
<input type="file" id="myFile" hidden accept="image/*" />
<button onclick="myFile.click()">繪製圖檔</button>
```

```
const img = new Image();
img.addEventListener("load", () => {
  ctx.drawImage(img, 0, 0);
  URL.revokeObjectURL(img.src); // 畫完就釋放暫存 URL 佔用的記憶體
});

myFile.addEventListener("change", () => {
  img.src = URL.createObjectURL(myFile.files[0]);
});
```

3.5 畫文字

`fillText()` 畫實心字、`strokeText()` 畫描邊字，常搭配 `textAlign` 與 `textBaseline` 控制錨點位置。

範例：307-canvas-text.html

```
ctx.fillStyle = "red";
ctx.strokeStyle = "blue";
ctx.lineWidth = 2;

ctx.textAlign = "center"; // start | end | left | right | center
ctx.textBaseline = "middle"; // top | hanging | middle | alphabetic(預設) |
ideographic | bottom

ctx.font = "bold 72px serif";
ctx.fillText("Hello Canvas", 300, 100);

ctx.font = "bold 72px arial";
```

```
ctx.strokeText("Hello Canvas", 300, 200);

ctx.fillText("哈囉", 300, 300);
```

`font` 屬性的格式跟 CSS 相同：

`[font-style] [font-weight] <font-size> <font-family>`。

畫中日韓文字時 `textBaseline` 可用 `ideographic`，基線會對齊表意文字的下緣，比 `alphabetic` 更貼合。

3.6 截取影片

`drawImage()` 第一個參數除了 `Image`，也吃 `<video>`、`<canvas>` 元素 — 用 `<video>` 就是擷取目前播放的那一幀。

範例：308-canvas-video.html

```
<video src="media/big-buck-bunny_trailer.webm" preload="auto" controls></video>
<button onclick="takePic()">截取</button>
<canvas id="myCanvas"></canvas>
```

```
const video = document.querySelector("video");

const takePic = () => {
  // 把 canvas 設為影片實際解析度，避免拉伸
  myCanvas.width = video.videoWidth;
  myCanvas.height = video.videoHeight;
  ctx.drawImage(video, 0, 0);
};
```

同源限制：跨網域影片會讓 canvas 變成「污染 (tainted)」，後續無法用 `toDataURL()` 取出影像。要解，影片伺服器必須回 `Access-Control-Allow-Origin`，且 `<video>` 加 `crossorigin="anonymous"`。

3.7 塗鴉練習

把滑鼠事件 + canvas 路徑連起來，就是一個簡易塗鴉板：`mousedown` 開新路徑、`mousemove` 連線並 `stroke`、`mouseup` 收尾。`toDataURL()` 把畫布內容轉成 base64 圖檔字串，常用於「下載」或存到 `localStorage`（後面 §7 會用到）。

範例：310-canvas-draw-app.html

```
<canvas id="myCanvas" width="600" height="400"></canvas>
<input type="color" id="pColor" value="#0000FF" />
<input type="number" id="pSize" min="1" max="50" step="1" value="5" />
```

```
<button onclick="clearCanvas()">清空</button>
<button onclick="saveCanvas()">下載 PNG</button>
```

```
const ctx = myCanvas.getContext("2d");
ctx.lineCap = "round";
ctx.lineJoin = "round";

myCanvas.addEventListener("mousedown", (e) => {
  ctx.beginPath();
  ctx.moveTo(e.offsetX, e.offsetY);
  myCanvas.addEventListener("mousemove", onMove);
  window.addEventListener("mouseup", onUp);
});

const onMove = (e) => {
  ctx.lineTo(e.offsetX, e.offsetY);
  ctx.stroke();
};
const onUp = () => {
  myCanvas.removeEventListener("mousemove", onMove);
  window.removeEventListener("mouseup", onUp);
};

// 跟著選色器/筆寬欄位變化
const setPen = () => {
  ctx.strokeStyle = pColor.value;
  ctx.lineWidth = pSize.value;
};
[pColor, pSize].forEach((el) => el.addEventListener("change", setPen));
setPen();

const clearCanvas = () =>
  ctx.clearRect(0, 0, myCanvas.width, myCanvas.height);

const saveCanvas = () => {
  const a = document.createElement("a");
  a.download = "draw.png";
  a.href = myCanvas.toDataURL("image/png");
  a.click();
};
```

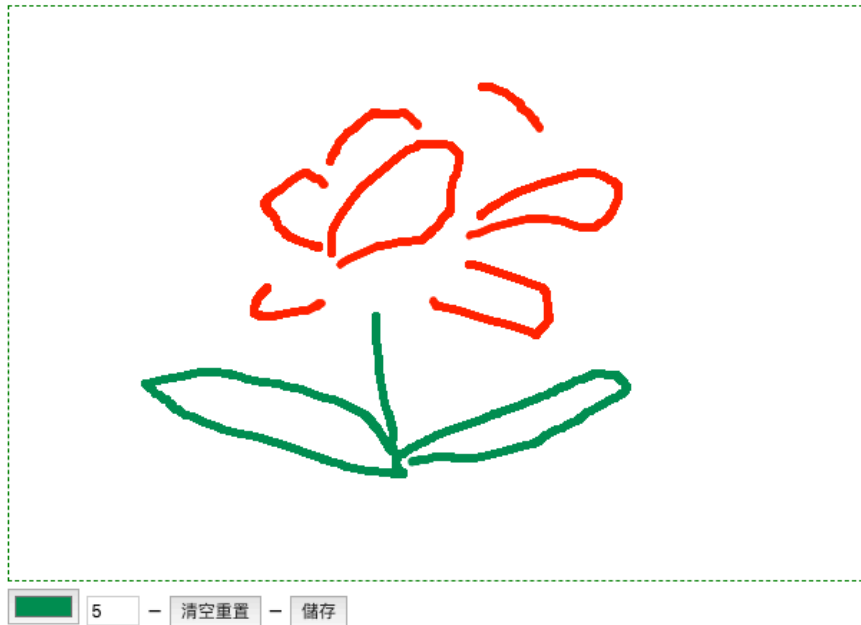


圖 5 Canvas 塗鴉應用

`mouseup` 掛在 `window` 而非 `canvas` — 這樣使用者拖到 `canvas` 外才放開時也能收到事件，否則塗鴉會卡住。

3.8 動畫

Canvas 沒有「移動物件」的概念，動畫就是不斷重畫：每一格先 `clearRect()` 清空、再依新的狀態把整個畫面畫一次。

範例：`312-goto-sleep.html` — 一圈圈往外擴散的同心圓，並示範把畫布撐滿整個視窗。

```
<canvas id="myCanvas"></canvas>
<script>
  const myCanvas = document.querySelector("#myCanvas");
  const ctx = myCanvas.getContext("2d");
  // 用 JS 設 HTML 屬性把畫布撐滿視窗（不是用 CSS）
  myCanvas.width = window.innerWidth;
  myCanvas.height = window.innerHeight;
  const cx = myCanvas.width / 2;
  const cy = myCanvas.height / 2;

  ctx.strokeStyle = "#878686";
  ctx.lineWidth = 20;

  let diff = 1;
  const myDraw = () => {
    let i = 10 + diff;
    i %= 30; // 半徑每次多 1，滿 30 就歸零，形成「持續往外」的錯覺
    for (; i < 1200; i += 30) {
      ctx.beginPath();
      ctx.arc(cx, cy, i, 0, Math.PI * 2);
      ctx.stroke();
    }
  }
  myDraw();
  setInterval(myDraw, 1000);
</script>
```

```

    }
};

setInterval(() => {
    diff += 1;
    ctx.clearRect(0, 0, myCanvas.width, myCanvas.height); // 先清空
    myDraw(); // 再整張重畫
}, 30);
</script>

```

範例用 `setInterval` 是為了直觀；正式動畫建議改用 `requestAnimationFrame(myDraw)`，瀏覽器會對齊螢幕更新率（通常 60 fps）、分頁切到背景時自動暫停，比固定間隔省電也更順。

範例編號 308 之後直接跳到 310、312，中間沒有 309、311，屬預留編號。

4. SVG

SVG (Scalable Vector Graphics) 是 W3C 標準的向量圖格式。HTML5 把它直接整合進 HTML，不需要 `<object>` 或 `<embed>`，可以直接寫 `<svg>` 標籤。每個圖形（`<circle>`、`<rect>`、`<polygon>` ...）都是真實的 DOM 節點，可以掛事件、套 CSS、在 DevTools 裡檢視，剛好和 Canvas「畫上去就化為像素」相反。

特性	SVG	Canvas
圖形模型	向量、保留物件樹	點陣、立即模式
縮放	無失真	放大會糊
個別元素操作	容易 (CSS / DOM API)	自己重畫
大量物件效能	較差 (DOM 多)	好
適合	圖示、圖表、插畫、UI 元件	遊戲、動畫、影像濾鏡

工作流：用 Illustrator、Inkscape、Figma 等繪圖工具畫好，匯出成 `.svg` 檔，內容就是純 XML，直接複製貼到 HTML 即可。

文件：<https://developer.mozilla.org/zh-TW/docs/Web/SVG/Reference/Element>

函式庫：[D3.js](#)（資料視覺化）、[SVG.js](#)（操作 SVG 的輕量 API）。

4.1 常見的 SVG 標籤

標籤	用途
<code><svg></code>	頂層容器，設定顯示區與座標系
<code><line></code>	線段
<code><rect></code>	矩形
<code><circle></code>	圓形
<code><ellipse></code>	橢圓
<code><polygon></code>	多邊形（封閉）
<code><polyline></code>	連續折線（不封閉）
<code><path></code>	萬用路徑（複雜形狀都靠它）
<code><g></code>	群組（要對多個元素一起套變形/樣式時用）
<code><text></code>	文字

viewBox 與座標系

`<svg>` 上的 `viewBox="0 0 300 200"` 定義「內部的座標範圍」是寬 300 × 高 200，跟外觀的顯示尺寸（CSS `width / height`）無關。瀏覽器會自動縮放映射 — 這就是 SVG「縮放不失真」的關鍵：寫的座標永遠固定，渲染時才換算成實際像素。

SVG 與 Canvas 都採「左上角原點、Y 軸向下」的座標系。

4.2 範例：互動式 SVG

每個 SVG 元素都是 DOM 節點，可以掛 `addEventListener`、套 CSS class、用 `:hover` 偽類，跟一般 HTML 元素一樣。

範例：`401-svg-demo.html`

```
<svg viewBox="0 0 300 200" xmlns="http://www.w3.org/2000/svg">
  <polygon class="c1" points="0,100 25,25 75,75 100,0" />
  <polygon
    points="100,150 125,75 175,125 200,50"
    fill="green"
    stroke="black"
    stroke-width="10"
  />
</svg>

<style>
  svg { width: 300px; height: 200px; }
  .c1 { fill: red; }
  .c1:hover { fill: blue; }
  .c2 { fill: orange; }
  .c2:hover { fill: purple; }
</style>
```

```
<script>
  // 點擊第一個 polygon 切換 class
  const polygon1 = document.querySelector("polygon");
  polygon1.addEventListener("click", () => {
    const next = polygon1.classList.contains("c1") ? "c2" : "c1";
    polygon1.setAttribute("class", next);
  });
</script>
```

幾個容易踩雷或值得注意的點：

- 樣式可以寫成屬性或 CSS：第一個 polygon 用 class 控制顏色（CSS 中 `fill: red`），第二個直接寫屬性 `fill="green"`。CSS 的優先權通常較高，方便在 hover 時改色。
- `className` 在 SVG 上行不通：HTML 元素可以 `el.className = "c2"`，但 SVG 元素必須用 `setAttribute("class", ...)` 或 `classList.add/remove`。
- `points` 屬性的格式：`x1,y1 x2,y2 ...`，逗號或空白都算分隔符，常見寫成 `"0,100 25,25 75,75 100,0"` 較易讀。

5. Drag and Drop（拖放）

HTML5 提供原生的拖放 API，比手刻 `mousedown / mousemove / mouseup` 簡單，而且支援跨視窗，甚至能從桌面拖檔案進瀏覽器（§6 會用到）。整個流程靠七個事件貫串，分屬「來源（被拖的元素）」與「目標（接收區）」兩端：

事件	端	觸發時機
<code>dragstart</code>	來源	開始拖（按住開始拖移）
<code>drag</code>	來源	拖移過程中持續觸發（類似 <code>mousemove</code> ，多數情境用不到）
<code>dragend</code>	來源	結束拖（放開或取消）
<code>dragenter</code>	目標	拖曳物進入目標範圍
<code>dragover</code>	目標	在目標範圍內移動，必須 <code>preventDefault()</code> 才會觸發 <code>drop</code>
<code>dragleave</code>	目標	拖曳物離開目標範圍
<code>drop</code>	目標	在目標範圍內放開

預設可拖：`<a>` 與 `<img>` 不必設定就能拖；其他元素要拖必須加 `draggable="true"` 屬性。

5.1 讓元素能被拖

最小可動範例 — 元素加 `draggable="true"`、聽 `dragstart` 與 `dragend` 確認事件有觸發。CSS 部分省略，直接看 JS 行為。

範例：501-drag.html

```
<div class="rect">
  <div class="ball" draggable="true">??</div>
</div>

<script>
  const b = document.querySelector(".ball");
  const handler = (e) => console.log(e.type);
  b.addEventListener("dragstart", handler);
  b.addEventListener("dragend", handler);
</script>
```

`drag` 事件每幾毫秒觸發一次，多數情境用 `dragstart` 取起點、`dragend` 處理結束就夠了。

5.2 拖到目標再放開

要做出「拖到指定區域才有反應」，必須來源與目標雙方搭配：來源觸發 `dragstart` / `dragend`，目標觸發 `dragenter` / `dragover` / `dragleave` / `drop`。

最常見的踩雷點：`dragover` 的瀏覽器預設行為會阻止 `drop`，所以一定要在 `dragover` 內呼叫 `e.preventDefault()`，否則放手時 `drop` 不會觸發。MDN 建議 `dragenter` 也一併 `preventDefault()`，某些舊版瀏覽器需要。

範例：502-drag-drop.html

```
<div class="rect">
  <div class="ball" draggable="true">??</div>
</div>
<div class="rect" style="left: 250px"></div>
```

```
const [, rect2] = document.querySelectorAll(".rect");

// 進入時改色，給使用者視覺回饋
rect2.addEventListener("dragenter", (e) => {
  e.currentTarget.style.backgroundColor = "#ffe666";
});
// 一定要 preventDefault，drop 才會觸發
rect2.addEventListener("dragover", (e) => e.preventDefault());
// 離開或放開都要還原顏色
rect2.addEventListener("dragleave", (e) => {
  e.currentTarget.style.backgroundColor = "#a2e666";
});
rect2.addEventListener("drop", (e) => {
```

```
e.currentTarget.style.backgroundColor = "#a2eff6";
console.log("drop");
});
```

5.3 用 dataTransfer 傳資料

drop 事件不會自動知道「使用者拖了哪個東西過來」 — 這個資訊要靠 `event.dataTransfer` 在來源端寫入、在目標端讀出。可以把它想成跨「來源 → 目標」的暫存剪貼簿。

範例：503-drag-drop.html — 把元素的 `id` 寫進 `dataTransfer`，drop 時再撈回原元素 append 到目標：

```
<div class="rect">
  <div class="ball" draggable="true" id="b77">77</div>
</div>
<div class="rect" style="left: 250px"></div>
```

```
const b = document.querySelector(".ball");
const [, rect2] = document.querySelectorAll(".rect");

b.addEventListener("dragstart", (e) => {
  e.dataTransfer.setData("text", e.currentTarget.id); // 把 id 當資料
});

rect2.addEventListener("dragover", (e) => e.preventDefault());
rect2.addEventListener("drop", (e) => {
  const tid = e.dataTransfer.getData("text");
  e.currentTarget.append(document.getElementById(tid)); // 撈回原元素，移到目標
});
```

`setData(format, value)` 的 `format` 常用 `"text/plain"` / `"text/html"` / `"text/uri-list"`，也能用自訂格式（如 `"application/x-my-app"`）以區分不同來源。範例裡的 `"text"` 是 `"text/plain"` 的舊別名（IE 時代留下的相容寫法），新程式建議直接寫 `"text/plain"`。

5.4 多個拖曳元素

當要拖的元素很多、又想直接搬移元素本身（不必透過 `id` 反查），用一個外層變數記下「目前正在拖的元素」最直接。 `forEach` 一次幫所有 `.ball` 與 `.rect` 掛事件。

範例：504-drag-drop.html

```
<div class="rect">
  <div class="ball" draggable="true">7</div>
  <div class="ball" draggable="true" style="background-color: red">8</div>
  <div class="ball" draggable="true" style="background-color: blue">9</div>
</div>
<div class="rect" style="left: 250px"></div>
```

```
let dragTarget = null; // 目前正在拖的元素

document.querySelectorAll(".ball").forEach((b) => {
  b.addEventListener("dragstart", (e) => (dragTarget = e.currentTarget));
  b.addEventListener("dragend", () => (dragTarget = null));
});

document.querySelectorAll(".rect").forEach((r) => {
  r.addEventListener("dragover", (e) => e.preventDefault());
  r.addEventListener("drop", () => {
    if (dragTarget) r.append(dragTarget); // 把元素搬到新容器
  });
});
```

dataTransfer vs 外層變數： `dataTransfer` 適合「跨頁面、跨瀏覽器視窗」的資料傳遞（瀏覽器自動序列化）；外層變數則簡單直接，適合單頁內的元素搬移。

5.5 手刻拖放（選讀）

原生 DnD API 的限制是：拖曳中的「殘影」由瀏覽器決定、拖曳過程無法即時改變其他元素的排列。要做「拖曳時清單即時讓位」這種效果，仍得回到 `mousedown` / `mousemove` / `mouseup` 自己算座標。範例專案的 511 — 517 是一組循序漸進的手刻版本，可對照參考：

檔案	內容
511-dnd-custom-01.html	用三個滑鼠事件拖動單一元素（記下按下點的偏移量）
512-dnd-custom-02.html	放開時計算與目標的距離，夠近就吸附過去
513-dnd-custom-03.html	多個元素共用同一組 handler
514-dnd-custom-04.html	拖曳時暫時改成 <code>position: absolute</code> ，放開後回到文件流
515-dnd-custom-05.html	用 placeholder 佔位做排序清單
516-dnd-custom-06.html	placeholder 隨滑鼠位置移動，其他項目即時讓位
517-dnd-custom-07.html	改用事件委派， <code>mousedown</code> 只掛在容器上

6. FileList、File、FileReader

瀏覽器基於安全性，JS 不能任意讀取使用者磁碟上的檔案 — 必須由使用者明確操作（在 `<input type="file">` 選檔、或從檔案管理員拖進瀏覽器），瀏覽器才會把檔案內容開放給 JS。HTML5 提供三個型別處理這件事：

型別	角色
<code>FileList</code>	類陣列集合，是 <code>input.files</code> 與 <code>dataTransfer.files</code> 的型別
<code>File</code>	單一檔案的描述（ <code>name</code> 、 <code>type</code> 、 <code>size</code> 、 <code>lastModified</code> ），繼承自 <code>Blob</code>
<code>FileReader</code>	真正讀取檔案內容的物件，提供 <code>readAsText</code> 、 <code>readAsDataURL</code> 等非同步方法

拿到 `File` 物件只是拿到「描述」，還不是內容；要讀內容必須再用 `FileReader`，或丟給 `URL.createObjectURL()` 取得暫存 URL（直接給 `<img>` / `<video>` 用）。也無法用 JS 取得使用者端的真實磁碟路徑。

6.1 取得 File 物件

兩種典型來源：

1. `<input type="file">` 的 `change` 事件 → `input.files`
2. 拖放：`drop` 事件的 `e.dataTransfer.files`

兩者型別都是 `FileList`。

從檔案欄位取得

`<input type="file">` 預設外觀陽春，常見作法是把它隱藏起來，用自己設計的按鈕或區塊呼叫 `input.click()` 觸發。

範例：601-file-field.html

```
<input type="file" id="fileField" hidden multiple accept="image/*" />
<div class="container" onclick="fileField.click()">點擊選取檔案</div>

<script>
  const container = document.querySelector(".container");
  const fileField = document.querySelector("#fileField");

  fileField.addEventListener("change", () => {
    for (const f of fileField.files) {
      // f 是 File 物件：name, type, size, lastModified
      container.innerText += `\n${f.name} (${f.size} bytes)`;
    }
  });
</script>
```

`accept="image/*"` 讓檔案對話框預設只列出圖檔（也可以寫 `.png, .jpg`、`text/*`、`application/pdf`）。它只是提示不是驗證：使用者可以在對話框切成「所有檔案」，拖放更是完全繞過它。真的要限制型別，前端用 `file.type` 檢查（見 602）、後端再驗一次（見 §10）。

`multiple` 讓使用者可以一次選多個檔案。

`FileList` 不是真的陣列，但可以用 `for...of`，或 `Array.from(input.files)` 轉成陣列再用 `map` / `filter`。

從拖放取得（從檔案管理員拖檔進來）

接上 §5 的拖放事件 — 這次「來源」是作業系統的檔案管理員，瀏覽器只負責處理「目標」端事件。`e.dataTransfer.files` 就是使用者拖進來的檔案清單。

範例：602-dnd-files.html

```
<div class="container" title="從檔案管理員拖拉檔案進來"></div>
```

```
const container = document.querySelector(".container");

container.addEventListener("dragover", (e) => e.preventDefault());
container.addEventListener("drop", (e) => {
  e.preventDefault(); // 若不擋掉，瀏覽器會直接開啟圖檔

  for (const f of e.dataTransfer.files) {
    if (!f.type.startsWith("image/")) continue;

    const url = URL.createObjectURL(f); // 同步取得 blob: 暫存 URL
    container.innerHTML += `
      <div class="box">
        
      </div>`;
  }
});
```

URL.createObjectURL(file)：同步呼叫，回傳一個 `blob:...` 開頭的暫存網址，可以直接餵給 `<img>`、`<video>`、`<a download>` 等元素。網址在分頁關閉前有效；不再需要時記得呼叫 `URL.revokeObjectURL(url)` 釋放記憶體。

6.2 FileReader：讀取檔案內容

`File` 物件本身只有 metadata，要讀「內容」必須用 `FileReader`。它是事件驅動的非同步 API：呼叫 `readAs*()` 方法後，內容讀完會觸發 `load` 事件，從 `event.target.result` 取出結果。

方法	結果型別	適用
<code>readAsText(file, encoding?)</code>	字串	文字檔（ <code>.txt</code> 、 <code>.csv</code> 、 <code>.json</code> ...）
<code>readAsDataURL(file)</code>	base64 字串 (<code>data:...;base64, </code>)	嵌入 HTML 的小圖、要傳到後端的圖
<code>readAsArrayBuffer(file)</code>	<code>ArrayBuffer</code>	二進位處理（解析、加密、編碼轉換）

讀文字檔

範例：`603-read-text.html`

```
<input type="file" id="fileField" hidden accept="text/*" />
<button onclick="fileField.click()">開啟一個文字檔</button>
<textarea class="container" rows="100" cols="50"></textarea>
```

```
const container = document.querySelector(".container");
const fileField = document.querySelector("#fileField");

fileField.addEventListener("change", () => {
  if (!fileField.files.length) return;

  const reader = new FileReader();
  reader.addEventListener("load", (e) => {
    container.value = e.target.result; // 結果是字串
  });
  reader.readAsText(fileField.files[0]);
});
```

讀圖檔 (轉 Data URL)

讀成 data URL 之後可以直接當 `<img>` 的 `src`。讀檔是非同步的，多檔時每個檔都要新建一個 `FileReader` (同一個 reader 連續呼叫會互相覆蓋)。

範例：604-read-images.html

```
fileField.addEventListener("change", () => {
  for (const file of fileField.files) {
    const reader = new FileReader();
    reader.addEventListener("load", (e) => {
      const img = new Image();
      img.height = 150;
      img.src = e.target.result; // base64 字串
      container.append(img);
    });
    reader.readAsDataURL(file);
  }
});
```

readAsDataURL vs URL.createObjectURL

比較項	readAsDataURL	URL.createObjectURL
同步性	非同步 (監聽 <code>load</code>)	同步
結果型式	base64 字串 (含資料)	blob:... 短網址 (指向資料)
大小	比原檔多約 33% (base64)	與原檔同 (不複製)
適合	要序列化或存進 storage	直接顯示，效能好

7. Web Storage

前端常用的「資料儲存在瀏覽器」技術有三種：

方案	容量	隨請求送出	適用
Cookie	4 KB	每次都送	後端要看的資料（登入 session、語系設定）
Web Storage	5–10 MB	不會	純前端用的資料（草稿、UI 狀態、暫存）
IndexedDB	數百 MB+	不會	結構化資料、大量資料、要做索引查詢

Web Storage 包含兩個物件：`localStorage` 與 `sessionStorage`，API 完全相同，差別在資料的存活範圍。兩者都採同源政策（same-origin policy），不同網域的資料互不可見；儲存格式只能是字串（要存物件得自己 `JSON.stringify` / `JSON.parse`）。

物件	存活範圍
<code>localStorage</code>	整個網域共用，永久保存（除非手動或 JS 清除）
<code>sessionStorage</code>	限同一個瀏覽器分頁/視窗，分頁關閉就消失

7.1 共用方法

兩個物件都實作 `Storage` 介面，成員完全相同：

成員	用途
<code>length</code>	屬性，取得資料筆數
<code>key(n)</code>	取得索引 n 的 key
<code>setItem(key, value)</code>	設定（value 必須是字串）
<code>getItem(key)</code>	讀取，找不到回傳 <code>null</code>
<code>removeItem(key)</code>	移除單筆
<code>clear()</code>	清空全部

storage 事件：同源的其他分頁改了 `localStorage`，本分頁的 `window` 會收到 `storage` 事件（`e.key`、`e.oldValue`、`e.newValue`），是跨分頁同步狀態最簡單的方法。注意自己分頁的修改不會觸發自己的事件。

容量超限：`setItem` 超過額度會拋出 `QuotaExceededError`，存大字串（如 base64 圖檔）時記得用 `try/catch` 包起來。

7.2 使用 localStorage

儲存資料

`localStorage` 只能存字串，所以複雜結構要先 `JSON.stringify`。實務上常以「字首 + 唯一識別」當 key（這裡用毫秒 timestamp `Date.now()`），達到「同一網站不同功能不互撞」的效果。

範例：`701-localStorage-setItem.html`（701–704 是同一組四頁範例，頁首互有導覽連結，講義只列與主題相關的程式碼）

```
<form name="form1">
  <input type="text" name="name" required /> 姓名<br />
```

```

<input type="number" name="age" required /> 年齡<br />
<input type="text" name="mobile" required pattern="09\d{2}-?\d{3}-?\d{3}" /> 手機
<br />
<input type="submit" value="儲存" />
</form>

```

```

const PREFIX = "friend-book;";

document.form1.onSubmit = (e) => {
  e.preventDefault();
  const t = Date.now();
  const data = {
    name: document.form1.name.value,
    age: document.form1.age.value,
    mobile: document.form1.mobile.value,
    time: t,
  };
  localStorage.setItem(PREFIX + t, JSON.stringify(data));
  document.form1.reset();
  alert("儲存完成!");
};

```

localStorage.setItem("count", 1) 不會出錯，但讀回來會是字串 "1"，要 Number(...) 才能參與運算。布林、物件也都得自己處理序列化。

讀取整份列表

要列出某個前綴的所有資料，得遍歷 localStorage.length 筆 → key(i) 取 key → getItem(key) 取值。

範例：702-localStorage-getItem.html

```

<table>
  <tbody id="the-list"></tbody>
</table>

```

```

const PREFIX = "friend-book;";
const tbody = document.querySelector("#the-list");

const showList = () => {
  let str = "";
  for (let i = 0; i < localStorage.length; i++) {
    const key = localStorage.key(i);
    if (!key.startsWith(PREFIX)) continue;

    const item = JSON.parse(localStorage.getItem(key));
    str += `<tr>
      <td>${item.name}</td>
      <td>${item.age}</td>
    `;
  }
  tbody.innerHTML = str;
};

```

```

        <td>${item.mobile}</td>
    </tr>`;
}
tbody.innerHTML = str;
};
showList();

```

加上刪除按鈕

每列加個刪除按鈕，呼叫 `removeItem(key)` 後重新渲染列表。

`onclick="removeItem('${key}')"` 是把 `key` 直接寫進 HTML 的快寫法（這裡 `key` 是程式產生的 timestamp，不會有引號或注入問題）。

但 `item.name`、`item.mobile` 是使用者輸入的內容，直接塞進 `innerHTML` 就有 XSS 風險（輸入 `<img src=x onerror=...>` 就會執行）。示範用途無妨，正式專案應改用 `textContent` 逐格填值，或先把 `<`、`>`、`"` 跳脫。

範例：703-localStorage-list.html

```

const showList = () => {
    let str = '';
    for (let i = 0; i < localStorage.length; i++) {
        const key = localStorage.key(i);
        if (!key.startsWith(PREFIX)) continue;

        const item = JSON.parse(localStorage.getItem(key));
        str += `<tr>
            <td><button onclick="removeItem('${key}')">X</button></td>
            <td>${item.name}</td>
            <td>${item.age}</td>
            <td>${item.mobile}</td>
        </tr>`;
    }
    tbody.innerHTML = str;
};
showList();

const removeItem = (key) => {
    if (!confirm("確定要移除?")) return;
    localStorage.removeItem(key);
    showList();
};

```

依時間排序

上面的列表順序是依 `localStorage.key(i)` 給的順序，不保證跟新增順序一致（瀏覽器內部實作可能是 hash 表）。要排序就先把資料收進陣列再 `sort`。

範例：704-localStorage-list-order-by.html

```

const showList = () => {
  const items = [];
  for (let i = 0; i < localStorage.length; i++) {
    const key = localStorage.key(i);
    if (!key.startsWith(PREFIX)) continue;
    const item = JSON.parse(localStorage.getItem(key));
    items.push({ ...item, key });
  }
  items.sort((a, b) => a.time - b.time); // 依時間升冪

  tbody.innerHTML = items.map((i) => `
    <tr>
      <td><button onclick="removeItem('${i.key}')">X</button></td>
      <td>${i.name}</td>
      <td>${i.age}</td>
      <td>${i.mobile}</td>
    </tr>`).join("");
};
showList();

```

儲存圖片

任何字串都能存，包括 `canvas.toDataURL()` 產生的 base64 圖檔字串。注意這會吃容量（一張小圖可能就上百 KB），總額度大約 5–10 MB 看瀏覽器。

範例：705-localStorage-draw-app.html — 在 §3.7 塗鴉板的基礎上，把「下載 PNG」換成「儲存到 localStorage」，並加一個下拉選單載入之前存的圖。塗鴉與 `clearCanvas()` 的部分與 §3.7 相同，這裡只列新增的程式碼。

```

<canvas id="myCanvas" width="600" height="400"></canvas>
<input type="color" id="pColor" value="#0000FF" />
<input type="number" id="pSize" min="1" max="50" step="1" value="5" />
<button onclick="clearCanvas()">清空</button>
<button onclick="saveCanvas()">儲存</button>
載入 <select id="listCombo" onchange="loadCanvas(this.value)"></select>

```

```

const PREFIX = "shinder-draw;;";

// 儲存目前 canvas 為 base64 圖檔字串
const saveCanvas = () => {
  const dataURL = myCanvas.toDataURL("image/png");
  localStorage.setItem(PREFIX + Date.now(), dataURL);
  renewList();
};

// 載入指定 key 的圖到 canvas
const loadCanvas = (key) => {
  if (!key) return;
  clearCanvas();
  const img = new Image();
  img.onload = () => ctx.drawImage(img, 0, 0);

```

```

    img.src = localStorage.getItem(key);
  };

  // 從 localStorage 列出所有符合前綴的 key，更新下拉選單
  const renewList = () => {
    const items = [];
    for (let i = 0; i < localStorage.length; i++) {
      const key = localStorage.key(i);
      if (!key.startsWith(PREFIX)) continue;
      const ts = key.slice(PREFIX.length); // 去掉前綴就是 timestamp
      items.push({ key, date: new Date(Number(ts)).toLocaleString() });
    }
    listCombo.innerHTML =
      `<option value="">-- 選擇載入的項目 --</option>` +
      items.map((i) => `<option value="${i.key}">${i.date}</option>`).join("");
  };
  renewList();

```

7.3 使用 sessionStorage

sessionStorage API 跟 localStorage 完全一樣，差別只在「存活範圍」：限定於同一個瀏覽器分頁，分頁/視窗關閉資料就消失。同分頁內換頁、重新整理都還在。

範例：711-sessionStorage-1.html（寫入）

```

<div id="info"></div>
<a href="712-sessionStorage-2.html">next</a>
<script>
  // 顯示上次留下的值（重新整理才會看到）；info 是靠 id 自動成為全域變數
  info.innerHTML = sessionStorage.getItem("s-key");
  // 寫入新值
  sessionStorage.setItem("s-key", "Hello ~~");
</script>

```

範例：712-sessionStorage-2.html（讀取）

```

<div id="info"></div>
<script>
  info.innerHTML = sessionStorage.getItem("s-key"); // 讀到 711 寫入的值
</script>

```

驗證：從 711 點 next 到 712 能讀到值；直接打開 712 則讀不到（不同分頁、不同 session）。把 sessionStorage 換成 localStorage 試試，連新開一個瀏覽器視窗都能讀到。

8. Server-Sent Events (SSE)

Server-Sent Events 是一種 HTTP 回應格式 — 伺服器把回應保持開啟、間隔送出純文字訊息，瀏覽器透過 `EventSource` 物件接收。資料只能「伺服器 → 用戶端」單向流，但比 WebSocket 簡單（在 HTTP 上跑、不需要新 protocol）；也比輪詢（polling）省，因為連線只開一次。

比較	SSE	WebSocket	HTTP 輪詢
方向	伺服器 → 用戶端（單向）	雙向	單向（每次重發）
Protocol	HTTP（純文字）	ws/wss	HTTP
自動重連	內建	自己處理	—
資料型別	文字（UTF-8）	文字 + 二進位	任意
適合	通知、即時報價、AI 串流	聊天室、共同編輯、遊戲	不常變動的低頻次更新

8.1 伺服器送出格式

伺服器必須回這三個 HTTP 標頭：

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

訊息格式以 `欄位: 值\n` 表示，訊息與訊息之間用空行（`\n\n`）分隔：

```
: 可以放註解\n      ← 冒號開頭的行是註解，可拿來保活
event: myevent\n    ← 自訂事件名稱，省略時以 "message" 觸發
id: 20\n          ← 訊息 id；用戶端記住後重連時會放在 Last-Event-ID 標頭送回
retry: 5000\n      ← 連線中斷後重連等待毫秒數（未指定時由瀏覽器決定，多為 3-5 秒）
data: 傳送的資料\n\n ← 實際資料；多行就重複 data: 行
```

8.2 伺服器：每兩秒推一次時間

用 Express 寫一個簡單的 SSE 端點，用 `setInterval` 持續送目前時間。**重點**：用戶端關閉連線時要清掉 timer，否則 timer 會繼續跑、記憶體與 socket 都會洩漏。

```
// index.js
app.get("/try-sse", (req, res) => {
  res.writeHead(200, {
    "Content-Type": "text/event-stream",
    "Cache-Control": "no-cache",
    Connection: "keep-alive",
  });

  let id = 30;
  const timer = setInterval(() => {
    res.write(`id: ${id++}\n`);
    res.write(`data: ${new Date().toLocaleString()}\n\n`);
  }, 2000);
```

```
// 用戶端關閉分頁/離開時清掉 timer
req.on("close", () => clearInterval(timer));
});
```

這一節起的範例都要透過 Express 開啟 (`http://localhost:3031/901-sse.html`)，直接點兩下 HTML 檔用 `file://` 開是連不到 `/try-sse` 的。

8.3 用戶端：用 EventSource 接收

EventSource 物件自帶連線與自動重連，三個關鍵事件：

事件	觸發時機
open	連線建立
message	收到沒有 event: 欄位的訊息（也是預設事件名）
error	連線中斷或失敗（會自動嘗試重連，不必自己重新 new）

範例：`901-sse.html`

```
<div id="info"></div>
<script>
  const info = document.querySelector("#info");
  const es = new EventSource("/try-sse");

  es.addEventListener("open", () => {
    info.style.backgroundColor = "orange"; // 連上時改色
  });

  es.addEventListener("message", (e) => {
    // e.lastEventId 對應伺服器送的 id 欄位
    info.innerHTML = `${e.lastEventId}: ${e.data}`;
  });

  es.addEventListener("error", (e) => {
    console.log("error", e); // error 後瀏覽器會自動重連
  });
</script>
```

幾個常用的延伸：

```
// 伺服器送 event: myevent 時，用自訂事件名接收（不會進 message）
es.addEventListener("myevent", (e) => console.log(e.data));

// 主動關閉；關掉後不會再自動重連
es.close();
```

同源限制： `EventSource` 預設只能連同源 URL；要跨域必須伺服器回 CORS 標頭，並在前端用 `new EventSource(url, { withCredentials: true })`。

連線數： 瀏覽器對同網域的 SSE 連線數有限制（HTTP/1.1 通常 6 條）；HTTP/2 沒此限制。

9. WebSocket

WebSocket 在 HTTP 1.1 之上「升級」成持續開啟的雙向連線，伺服器可以主動推訊息給用戶端、用戶端也能即時回，常用於聊天室、即時客服、共同編輯、線上遊戲等需要雙向通訊的應用。和 §8 的 SSE 比，WebSocket 是雙向、可傳二進位；但 protocol 不是 HTTP，需要專屬的伺服器（這裡用 `ws` 套件）。

特性	WebSocket	SSE
方向	雙向	伺服器 → 用戶端
Protocol	<code>ws://</code> / <code>wss://</code>	HTTP
資料型別	文字 + 二進位	文字
自動重連	自己處理	內建
適合	聊天室、共編、遊戲	通知、串流、AI 回應

OSI 層級： WebSocket 在第 7 層應用層（HTTP 也是），底層的 Socket 在第 4 層傳輸層。Socket 太底層所以拿來做應用很煩瑣，WebSocket 等於是「為了 web 應用而生的 socket 抽象層」。

Port： `ws://` 用 80、`wss://` 用 443（實務上常自選 port，搭配反向代理）。

9.1 WebSocket 用戶端 API

瀏覽器內建 `WebSocket` 類別，事件驅動：

事件	觸發時機
<code>open</code>	連線建立
<code>message</code>	收到訊息（從 <code>event.data</code> 取資料）
<code>close</code>	連線關閉
<code>error</code>	發生錯誤

連線狀態 `socket.readyState`：

常數	值	描述
<code>CONNECTING</code>	0	連線尚未打開
<code>OPEN</code>	1	連線已打開，可進行通訊
<code>CLOSING</code>	2	連線正在關閉
<code>CLOSED</code>	3	連線已關閉或無法打開

實務上送資料前要檢查 `readyState === WebSocket.OPEN`：連線還在 `CONNECTING` 時呼叫 `send()` 會拋出 `InvalidStateError`；已經 `CLOSING` / `CLOSED` 則訊息被靜默丟棄、不會報錯。也可以查 `bufferedAmount` 屬性看送出佇列堆了多少 bytes 沒清空。

9.2 ws : Node.js WebSocket library

瀏覽器內建用戶端，但伺服器要自己用程式庫。Node.js 最常用 `ws`：

```
npm i ws
```

可以是獨立的 server（自己佔一個 port），也可以掛到既有的 Node `http/Express` server 上共用同一個 port。後面的範例採用後者 — `echo`、`chat`、`draw` 三條 path 共用 Express 的同一個 port，設定方式見 §9.3。

文件：<https://github.com/websockets/ws/blob/master/doc/ws.md>

9.3 設定：三條 WebSocket 共用同一個 port

`ws` 套件兩種啟動模式：

- **獨立 port**：`new WebSocketServer({ port: 3070 })`，自己跑一個獨立 server
- **跟 HTTP server 共用**：`new WebSocketServer({ noServer: true })` 加上 Express HTTP server 的 `upgrade` 事件分流

下面三節的 `echo` / `chat` / `draw` 都用後者，三條 path 共用 Express 的同一個 port。理由：CORS 設定點少（前後端同源）、反向代理 / cloudflared tunnel 只要 forward 一個 port、防火牆友善。

`index.js` 啟動骨架：

```
import express from "express";
import { WebSocketServer } from "ws";
import setupEcho from "./routes/ws-echo.js";
import setupChat from "./routes/ws-chat.js";
import setupDraw from "./routes/ws-draw.js";

const web_port = process.env.WEB_PORT || 3031;
const app = express();
// ... 其他 routes (SSE、上傳、Ollama chat 等)

const server = app.listen(web_port, () => {
  console.log(`伺服器啟動於通訊埠：http://localhost:${web_port}`);
});

// 三個 WebSocketServer，noServer 模式（不自己起 listener）
const wss = {
  echo: new WebSocketServer({ noServer: true }),
  chat: new WebSocketServer({ noServer: true }),
```

```

    draw: new WebSocketServer({ noServer: true }),
  });

  setupEcho(wss.echo);
  setupChat(wss.chat);
  setupDraw(wss.draw);

  // HTTP upgrade 事件 → 依 path 分流
  server.on("upgrade", (req, socket, head) => {
    const { pathname } = new URL(req.url, `http://${req.headers.host}`);
    const key = pathname.replace(/^\/ws\/$/, "");
    // 用 hasOwnProperty 查表，避免 /ws/constructor 這類路徑取到原型鏈上的成員
    const target = Object.prototype.hasOwnProperty.call(wss, key) ? wss[key] : null;
    if (!target) {
      socket.destroy();
      return;
    }
    target.handleUpgrade(req, socket, head, (ws) => {
      target.emit("connection", ws, req);
    });
  });
});

```

每支 `routes/ws-*.js` 改成 `export` 一個 `setup(wss)` 函式，把 `connection` listener 掛在從外面注入的 `wss` 上。

為什麼要 `Object.prototype.hasOwnProperty`： `wss` 是一般物件，直接 `wss[key]` 查表時， `key` 若是 `constructor`、`toString` 這類名稱會查到 `Object.prototype` 上的函式， `!target` 判斷不到，接著呼叫 `handleUpgrade` 就拋出未捕捉的例外，整個 Node 行程會掛掉。任何「使用者輸入當 `key` 查物件」的地方都要防這一手，或改用 `Map`。

9.4 回音程式

最簡單的 WebSocket 應用：用戶端送什麼，伺服器原樣送回。

伺服器端 — `routes/ws-echo.js`：

```

export default function setup(wss) {
  wss.on("connection", (ws, req) => {
    const ip = req.socket.remoteAddress;
    ws.send(`連線了，你來自 ${ip}，目前連線人數: ${wss.clients.size}`);

    ws.on("message", (message) => {
      ws.send(message.toString()); // 原訊息送回
    });
  });
}

```

`message` 在 `ws` 套件預設是 `Buffer`，要 `.toString()` 才能轉成字串。

前端 — A03-WebSocket-echo.html :

```
<button id="connectBtn">連線</button>
<input type="text" id="myInput" />
<button id="sendBtn" disabled>送出</button>
<div id="info"></div>

<script>
  let socket;

  connectBtn.onclick = () => {
    if (socket?.readyState === WebSocket.OPEN) return;

    socket = new WebSocket(`ws://${location.host}/ws/echo`);
    socket.addEventListener("open", () => (sendBtn.disabled = false));
    socket.addEventListener("close", () => (sendBtn.disabled = true));
    socket.addEventListener("message", (e) => {
      info.innerHTML += `${e.data}<br>`;
    });
  };

  sendBtn.onclick = () => socket.send(myInput.value);
</script>
```

`location.host` 已經包含 port (如 `localhost:3031`)。HTTPS 環境記得改 `wss://`。這裡沒有在 `send` 前檢查 `readyState`，是靠「連上才啟用送出按鈕」(`disabled`) 擋住未連線狀態；§9.7 的塗鴉板則示範顯式檢查。

9.5 Map 和 Set 型別

下一節的聊天室要用到 `wss.clients`，先補一個前提：`ws` 套件的 `WebSocketServer.clients` 屬性是 `Set` 型別，存放所有目前連線的用戶端。`Map` 與 `Set` 是 ES6 引進的集合型別：

集合	角色
Map	鍵值對集合，類似 <code>Object</code> ，但 <code>key</code> 可以是任何型別（含物件、函式）
Set	值的集合，自動去重，沒有索引

範例：A01-Map.html

```
const obj = { name: "bill" };
const f1 = () => {};
const map = new Map(); // 比 Object 強：key 可以是物件、函式

map.set({}, 123); // 兩個不同的空物件 → 兩筆不同 entry
map.set({}, 456);
map.set(obj, "abc");
map.set(obj, [1, 2, 3]); // 同一個 obj key → 覆寫舊值
map.set(f1, "函式");
```

```

console.log(map.size); // 4
console.log(map.get(obj)); // [1, 2, 3]
console.log([...map.keys()]); // 攤平成陣列

map.forEach((value, key) => console.log({ value, key }));

```

範例：A02-Set.html

```

const set = new Set();
const obj = { name: "bill" };
const f1 = () => {};

set.add(obj);
set.add(obj); // 同個 obj → 不會增加
set.add(f1);
console.log(set.size); // 2

console.log([...set.keys()]); // 攤平成陣列
set.forEach((v) => console.log(v));

```

比 Object 多了什麼：Object 的 key 只能是字串或 Symbol；用物件當 key 會被自動 `.toString()` 變成 `"[object Object]"`，不同物件會撞在一起。Map 解決這個問題，也保證走訪順序就是插入順序。

9.6 簡易聊天室

聊天室是「廣播」模式 — 任一個用戶端送訊息，伺服器轉送給所有連線中的用戶端。

`wss.clients` 就是這個「所有人」清單。第一次傳入的訊息當作使用者名稱，之後的訊息加上名稱前綴後廣播。

伺服器端 — `routes/ws-chat.js`：

```

import WebSocket from "ws";

export default function setup(wss) {
  const broadcast = (msg) => {
    wss.clients.forEach((c) => {
      if (c.readyState === WebSocket.OPEN) c.send(msg);
    });
  };

  wss.on("connection", (ws, req) => {
    const user = { name: "", ip: req.socket.remoteAddress };

    ws.on("message", (message) => {
      const m = message.toString();
      if (!user.name) {
        // 第一次進來：當作報上名字
        user.name = m;
      }
    });
  });
}

```

```

    broadcast(`${user.name}(${user.ip}) 進入，目前 ${wss.clients.size} 人`);
  } else {
    broadcast(`${user.name}: ${m}`);
  }
});

ws.on("close", () => {
  if (user.name) {
    broadcast(`${user.name}(${user.ip}) 離開，目前 ${wss.clients.size} 人`);
  }
});
});
}

```

前端 — `A04-WebSocket-chat.html` (與 `echo` 結構幾乎相同，差別在用 `form` 包輸入欄支援 Enter 送出)：

```

<button id="connectBtn">連線</button>
<form id="sendForm">
  <input type="text" id="myInput" />
  <button id="sendBtn" disabled>送出</button>
</form>
<div id="info"></div>

<script>
  let socket;

  connectBtn.onclick = () => {
    if (socket?.readyState === WebSocket.OPEN) return;
    socket = new WebSocket(`ws://${location.host}/ws/chat`);
    socket.addEventListener("open", () => (sendBtn.disabled = false));
    socket.addEventListener("close", () => (sendBtn.disabled = true));
    socket.addEventListener("message", (e) => (info.innerHTML += `${e.data}<br>`));
  };

  sendForm.addEventListener("submit", (e) => {
    e.preventDefault();
    socket.send(myInput.value);
    myInput.value = "";
  });
</script>

```

9.7 共用塗鴉板

把 §3.7 的塗鴉應用接上 WebSocket — 每段線條傳成 JSON，伺服器廣播給所有連線。多人開啟同個頁面會看到同一張畫布同步繪製。

伺服器端 — `routes/ws-draw.js` (廣播邏輯與聊天室相同)：

```

import WebSocket from "ws";

```

```
export default function setup(wss) {
  wss.on("connection", (ws) => {
    ws.on("message", (message) => {
      wss.clients.forEach((c) => {
        if (c.readyState === WebSocket.OPEN) c.send(message.toString());
      });
    });
  });
}
```

前端 — A05-WebSocket-draw.html (HTML 與 §3.7 相同，塗鴉部分只列與 §3.7 不同的地方)：

```
<canvas id="myCanvas" width="600" height="400"></canvas>
<input type="color" id="pColor" value="#0000FF" />
<input type="number" id="pSize" min="1" max="50" step="1" value="5" />
<button onclick="clearCanvas()">清空重置</button>
```

```
const ctx = myCanvas.getContext("2d");
ctx.lineCap = "round";
ctx.lineJoin = "round";
```

```
const socket = new WebSocket(`ws://${location.host}/ws/draw`);
```

// 收到別人畫的線就在自己 canvas 上重畫

```
socket.addEventListener("message", (e) => {
  try {
    const d = JSON.parse(e.data);
    ctx.beginPath();
    ctx.strokeStyle = d.c;
    ctx.lineWidth = d.s;
    ctx.moveTo(d.x1, d.y1);
    ctx.lineTo(d.x2, d.y2);
    ctx.stroke();
  } catch (ex) {}
});
```

// 自己畫線時：不直接 stroke，而是送 JSON 給伺服器

```
const startP = { x: 0, y: 0 };
```

```
myCanvas.addEventListener("mousedown", (e) => {
  startP.x = e.offsetX;
  startP.y = e.offsetY;
  myCanvas.addEventListener("mousemove", onMove);
  window.addEventListener("mouseup", onUp);
});
```

```
const onMove = (e) => {
  const data = {
    x1: startP.x, y1: startP.y,
    x2: e.offsetX, y2: e.offsetY,
```

```

    c: pColor.value,
    s: pSize.value,
  };
  if (socket.readyState === WebSocket.OPEN) {
    socket.send(JSON.stringify(data));
  }
  startP.x = e.offsetX;
  startP.y = e.offsetY;
};
const onUp = () => {
  myCanvas.removeEventListener("mousemove", onMove);
  window.removeEventListener("mouseup", onUp);
};

```

設計要點：自己畫線時不直接呼叫 `ctx.stroke()`，而是送 JSON 給伺服器 — 伺服器廣播後自己也會收到 `message` 事件，這時才畫到 canvas。所有人的畫線順序由伺服器決定，畫面才會一致。

練習：範例的「清空重置」按鈕只清自己的畫布、不會廣播。試著把清空也做成一種訊息（例如 `{ action: "clear" }`）送給伺服器，讓所有人同步清空。

10. AJAX 上傳檔案

傳統的 `<form method="POST" enctype="multipart/form-data">` 提交檔案會重新整理整個頁面、不能在送出前預覽、也無法用 JS 處理回應。HTML5 用 `FormData` + `fetch` 把表單序列化成 multipart 內容，配合 `<input type="file">` 與 §6 的 `URL.createObjectURL`，可以做到「選了就預覽」「不重新整理就上傳」。

`fetch` 本身沒有上傳進度事件。要做進度條得用 `XMLHttpRequest` 的 `upload.onprogress`，或改用 Streams API 分段送出。

伺服器端用 `multer` 中介軟體解析 multipart 內容，把欄位填進 `req.body`、檔案放進 `req.file` 或 `req.files`。

```

npm i multer uuid
mkdir -p public/images # 上傳的圖檔放這裡

```

`multer` 請用 2.x，1.x 已停止維護且有已知弱點；網路上許多舊教學仍是 1.x 寫法，API 大致相同。`uuid` 也可以用 Node 內建的 `crypto.randomUUID()` 取代，少裝一個套件。

10.1 後端：multer 設定模組

把 `multer` 設定獨立成模組重複使用。重點：

- `fileFilter`：拒絕不在白名單的 MIME type
- `diskStorage`：把檔案寫到硬碟（另一種選擇是 `memoryStorage`，全讀進記憶體再自己處理）
- 檔名隨機化：用 `uuid` 避免兩個使用者上傳同名檔案彼此覆寫

`routes/upload-img-module.js`：

```
import multer from "multer";
import { v4 as uuid } from "uuid";

// MIME type 對應到副檔名（同時也是白名單）
const extMap = {
  "image/jpeg": ".jpg",
  "image/png": ".png",
  "image/webp": ".webp",
};

const fileFilter = (req, file, cb) => {
  // 不在白名單的 MIME type 直接捨棄 (cb 第二個參數 false = 不收)
  cb(null, !!extMap[file.mimetype]);
};

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, "public/images"),
  filename: (req, file, cb) => cb(null, uuid() + extMap[file.mimetype]),
});

export default multer({ fileFilter, storage });
```

也可以加 `limits: { fileSize: 2 * 1024 * 1024 }` 限制單檔大小（這裡是 2 MB）。
`cb(null, false)` 是靜默捨棄：檔案不會存、`req.file` 會是 `undefined`，但請求本身照常成功，前端看到的仍是「上傳完成」。要讓使用者知道被拒，改成
`cb(new Error("不支援的檔案型別"))`，再用 Express 的 error middleware 回 400。

10.2 單檔上傳：個人資料 + 大頭貼

情境：表單包含姓名、簡介（純文字欄位）和大頭貼（檔案）。後端把資料合併存到 `profile.json`、檔案存到 `public/images/`、回傳整份資料給前端。

後端路由

`upload.single("avatar")` 是 `multer` 中介軟體：把表單裡 `name="avatar"` 的檔案放進 `req.file`，其他文字欄位照常進 `req.body`。

```
// index.js
import fs from "node:fs/promises";
import upload from "../routes/upload-img-module.js";

app.post("/uploads/profile", upload.single("avatar"), async (req, res) => {
  // 讀取既有資料（不存在就用空物件）
```

```

let data = {};
try {
  data = JSON.parse(await fs.readFile("./public/profile.json", "utf8"));
} catch {} // 檔案不存在或內容錯誤都算空

// 合併新資料
data = { ...data, ...req.body };
if (req.file) {
  data.avatar = "/images/" + req.file.filename;
}

try {
  await fs.writeFile("./public/profile.json", JSON.stringify(data));
  res.json({ success: true, data });
} catch (ex) {
  res.json({ success: false, data });
}
});

```

前端

三個重點：

1. 載入時 `fetch` 既有資料填回欄位（讓使用者看到目前設定）
2. 選檔即預覽：用 `URL.createObjectURL()` 取得本機暫存 URL 直接餵給 `<img>`，不必先上傳
3. 送出：用 `new FormData(form)` 把整個表單序列化丟給 `fetch` — 千萬別自己設 `Content-Type`，瀏覽器會自動加上含 boundary 的 `multipart/form-data` 標頭

B01-upload-single.html：

```

<form id="form1">
  <label>姓名 <input type="text" name="user" id="user" /></label><br />
  <label>大頭貼 <input type="file" name="avatar" id="avatar" accept="image/*" /></label><br />
  <img id="avatarImg" width="300" /><br />
  <label>簡介 <textarea name="description" id="description" cols="50"></textarea></label><br />
  <button type="submit">上傳</button>
</form>

```

```

// 1. 載入時取既有資料填回欄位
fetch("/profile.json")
  .then((r) => (r.ok ? r.json() : {}))
  .then((obj) => {
    user.value = obj.user || "";
    description.value = obj.description || "";
    if (obj.avatar) avatarImg.src = obj.avatar;
  });

```

```

// 2. 選檔即預覽
avatar.addEventListener("change", () => {

```

```

    if (avatar.files[0]) {
      avatarImg.src = URL.createObjectURL(avatar.files[0]);
    }
  });

// 3. 送出表單
form1.addEventListener("submit", async (e) => {
  e.preventDefault();
  const res = await fetch("/uploads/profile", {
    method: "POST",
    body: new FormData(form1),
  });
  const result = await res.json();
  alert(result.success ? "上傳完成" : "上傳失敗");
});

```

10.3 多檔上傳：相簿

情境：選一批圖片直接上傳，後端回傳每張圖的網址，前端立刻顯示。 `upload.array("photos")` 會把欄位的多個檔案放進 `req.files`（陣列）。

後端路由

```

// index.js
app.post("/uploads/photos", upload.array("photos"), (req, res) => {
  const urls = req.files.map((f) => "/images/" + f.filename);
  res.json(urls);
});

```

前端

沒有送出按鈕； `<input type="file" multiple>` 一旦選檔就觸發 `change` 事件、立即上傳。

B02-upload-multiple.html：

```

<form id="form1">
  <label>選了就上傳
    <input type="file" name="photos" id="photos" accept="image/*" multiple />
  </label>
  <div id="imgContainer"></div>
  <textarea id="description" cols="80" rows="10" readonly></textarea>
</form>

```

```

photos.addEventListener("change", async () => {
  const res = await fetch("/uploads/photos", {
    method: "POST",
    body: new FormData(form1),
  });
  const urls = await res.json();

  description.value = JSON.stringify(urls, null, 4);
});

```

```
imgContainer.innerHTML = urls
  .map((url) => `

```

不想用 `<form>` + `FormData(form)` 的話也能手動建：

```
const fd = new FormData();
for (const f of photos.files) fd.append("photos", f);
```

重點是 `name` 要跟 `upload.array("photos")` 對應。反過來說，範例裡的 `<textarea id="description">` 沒有 `name`，所以不會被 `FormData(form1)` 帶上，它只是用來顯示結果。

11. Web Worker

JavaScript 在瀏覽器只有一條主執行緒（main thread），UI 渲染、事件處理、所有 JS 都跑在這上面。一旦有耗時運算（圖像處理、加密、解壓、計算質數...），主執行緒就被卡住，整個頁面變得無法捲動、按鈕點不動。**Web Worker** 把運算搬到背景執行緒，主執行緒繼續流暢地處理 UI。

型態	適用情境
Dedicated Worker	一個頁面專屬一個 Worker（最常見）
Shared Worker	多個同源頁面共享一個 Worker（共用快取或連線）
Service Worker	PWA、離線快取、推播（見 §12）

通訊協定都是 `postMessage()` + `message` 事件，資料是 **structured clone**（深複製），所以兩邊的物件不共享記憶體。Worker 內可以用 `fetch`、`setTimeout`、`IndexedDB`，但 **不能存取** `document`、`window` 或 `localStorage`。

文件：https://developer.mozilla.org/zh-TW/docs/Web/API/Web_Workers_API/Using_web_workers

11.1 Dedicated Worker

用「計算 5,000 萬以內的所有質數」當示範 — 這是個典型的 CPU-bound 運算，跑下去 UI 會卡個幾秒，正好能對比有 Worker / 沒 Worker 的差異。

沒用 Worker（會卡）

按下「開始算質數」後，輸入欄完全無法打字、按鈕也不會立刻消失 — 主執行緒被吃滿。

範例：`C01-no-dedicated-worker.html`

```

<input type="text" placeholder="測試輸入欄" />
<button onclick="start(event)">開始算質數</button>
<div id="info"></div>

<script>
  const getPrimeNumbers = (num = 5e7) => {
    const startTime = Date.now();
    const pn = [2];
    for (let n = 3; n <= num; n++) {
      const m = Math.sqrt(n);
      let isPrime = true;
      for (const p of pn) {
        if (p > m) break;
        if (n % p === 0) { isPrime = false; break; }
      }
      if (isPrime) pn.push(n);
    }
    return { length: pn.length, howLong: Date.now() - startTime };
  };

  const start = (event) => {
    event.target.style.display = "none";
    const { length, howLong } = getPrimeNumbers();
    info.innerHTML = `${length} 個質數, ${howLong} 毫秒`;
  };
</script>

```

用 Worker (不卡)

主程式只負責「啟動 worker、收結果」，運算搬到 `js/dedicated-worker-prime-numbers.js`。

算質數時打字、滾動頁面都流暢。

範例：`C02-with-dedicated-worker.html`

```

<input type="text" placeholder="測試輸入欄" />
<button onclick="start(event)">開始算質數</button>
<div id="info"></div>

<script>
  const worker = new Worker("js/dedicated-worker-prime-numbers.js");

  worker.onmessage = (e) => {
    const { length, howLong } = e.data;
    info.innerHTML = `${length} 個質數, ${howLong} 毫秒`;
  };

  const start = (event) => {
    event.target.style.display = "none";
    worker.postMessage("start"); // 觸發 worker 開始算
  };
</script>

```

Worker 內部 — `js/dedicated-worker-prime-numbers.js` :

```
const getPrimeNumbers = (num = 5e7) => {
  const startTime = Date.now();
  const pn = [2];
  for (let n = 3; n <= num; n++) {
    const m = Math.sqrt(n);
    let isPrime = true;
    for (const p of pn) {
      if (p > m) break;
      if (n % p === 0) { isPrime = false; break; }
    }
    if (isPrime) pn.push(n);
  }
  return { length: pn.length, howLong: Date.now() - startTime };
};

// self 在 worker 裡是 DedicatedWorkerGlobalScope
self.onmessage = (event) => {
  if (event.data === "start") {
    postMessage(getPrimeNumbers());
  }
};
```

為什麼不把 `primes` 陣列傳回去：5,000 萬以內有約 300 萬個質數，整包 `postMessage` 要做 structured clone 的時間可能比運算本身還久。只回必要的 `length` 與 `howLong` 就好。

11.2 Shared Worker

Shared Worker 在多個同源頁面之間共用一份。只要其中一個頁面沒關，worker 就一直存在；新打開的頁面可以接續使用 worker 內已經有的狀態（例如已計算好的結果、已建立的 WebSocket 連線、共用的快取）。

兩個頁面 `C03-shared-worker-1.html` 與 `C04-shared-worker-2.html` 連到同一支 worker：

```
<input type="text" placeholder="測試輸入欄" />
<div id="info"></div>

<script>
  const worker = new SharedWorker("js/shared-worker-prime-numbers.js");

  // 注意：通訊不是直接 worker.postMessage，而是透過 worker.port
  worker.port.onmessage = (e) => {
    const { primeNumbers, runningTime, action } = e.data;
    const last = primeNumbers[primeNumbers.length - 1];
    info.innerHTML = `${action} - 最大質數: ${last}, ${runningTime} msec`;
  };
</script>
```

```
worker.port.postMessage("page-1"); // C04 改送 "page-2"
</script>
```

Worker 內部 — `js/shared-worker-prime-numbers.js` :

```
const pn = [2];
let runningTime;

// 載入時就算一次（兩個頁面共用同一份結果，只算這次）
const getPrimeNumbers = (num = 5e7) => {
  const startTime = Date.now();
  for (let n = 3; n <= num; n++) {
    const m = Math.sqrt(n);
    let isPrime = true;
    for (const p of pn) {
      if (p > m) break;
      if (n % p === 0) { isPrime = false; break; }
    }
    if (isPrime) pn.push(n);
  }
  runningTime = Date.now() - startTime;
};
getPrimeNumbers();

// 每個連入的頁面會觸發一次 onconnect; event.ports[0] 是該連線的專屬 port
self.onconnect = (event) => {
  const port = event.ports[0];
  port.onmessage = (e) => {
    port.postMessage({ primeNumbers: pn, runningTime, action: e.data });
  };
};
```

驗證：先開 C03 等它算完顯示結果，再開 C04 — 你會發現 C04 「秒回」（不必重算），因為兩個頁面共用同一份 `pn`。

除錯：Chrome 看不到 Shared Worker 在哪個分頁，要透過 `chrome://inspect/#workers` 檢視與下中斷點。

port.start()：範例用 `port.onmessage = ...` 會隱含啟動 port；若改成 `port.addEventListener("message", ...)`，必須另外呼叫 `port.start()`，否則永遠收不到訊息。

相容性：Chrome for Android 不支援 `SharedWorker`，行動裝置上要有 fallback。

12. PWA (Progressive Web App)

PWA 是把網頁應用包裝成「像原生 App 一樣」的技術組合 — 可以安裝到桌面或主畫面、有自己的圖示、支援離線、能推播通知；但底層仍然是 HTML/JS/CSS，沒有 App Store 審核流程，URL 一改就更新。

12.1 PWA 解決什麼問題

過去網頁與原生 App 之間有幾道明顯的鴻溝：

項目	原生 App	傳統網頁	PWA
安裝門檻	商店下載、佔空間	不需要	一鍵加入主畫面
啟動入口	主畫面圖示	瀏覽器書籤	主畫面圖示
離線可用	通常可	可（需 HTTPS + Service Worker）	可（同樣靠 Service Worker 快取）
更新流程	商店審核 + 使用者手動更新	URL 變了就生效	URL 變了就生效
推播	內建	可（需 HTTPS；iOS 上不行）	可（Push API；iOS 需先加到主畫面）
跨平台	要寫兩套（iOS / Android）	自然跨平台	自然跨平台

對開發者最大的價值：一份程式碼跑全平台、不必透過商店、更新隨網頁部署即時生效。

離線快取與推播其實是 Service Worker 與 Push API 的能力，只要 HTTPS 就能用，不需要「安裝」成 PWA；範例專案的 `F01-notification.html` 就是一般網頁直接發通知的例子。PWA 真正多出來的是「安裝入口」，以及在 iOS 上取得推播資格。

12.2 各平台支援情況

平台	安裝（加到主畫面）	Service Worker 快取	推播通知
Chrome / Edge（桌面）	全支援	全支援	全支援
Chrome（Android）	全支援	全支援	全支援
Firefox（桌面）	不支援（實驗中）	全支援	全支援
Firefox（Android）	全支援	全支援	全支援
Safari（iOS 16.4+）	全支援	全支援	須先加到主畫面
Safari（macOS Sonoma+）	全支援	全支援	全支援

HTTPS 強制：除了 `localhost`，PWA 一律必須以 HTTPS 提供，否則 Service Worker 註冊會直接失敗。

iOS 的推播：只有「加到主畫面」後開啟的 PWA 能收 Web Push；在 Safari 分頁內開的網頁收不到。這是 iOS 與其他平台最大的差異。

這張表最容易過時，實際支援度請以 <https://caniuse.com/> 為準。

12.3 三個關鍵組件

1. **Web App Manifest**（`manifest.webmanifest` JSON 檔）— 描述 App 名稱、圖示、起始 URL、顯示模式
2. **Service Worker**（背景執行的 JS 檔）— 攔截網路請求、實作快取與離線、處理推播

3. **Icons**（圖示檔）— 桌面 / 主畫面 / 啟動畫面用，至少要有 192×192 和 512×512（也可用 SVG 一張通用）

下面四個範例會逐步建出一個最簡 PWA。

12.4 Web App Manifest（宣告 App 身分）

`manifest.webmanifest` 是一份 JSON，告訴瀏覽器「我是一個 App，叫什麼名字、長什麼樣」。<link rel="manifest"> 把它接到 HTML 頁面。manifest 是可安裝的**必要條件但不是充分條件**，Chrome 還要求註冊 Service Worker（12.5）才會真的跳出安裝提示；N01 這一步只先讓 DevTools 讀得到 manifest。

範例：`N01-pwa-basic.html`

```
<!DOCTYPE html>
<html lang="zh">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>HTML5 API 學習 PWA</title>
    <link rel="manifest" href="manifest.webmanifest" />
    <meta name="theme-color" content="#3367d6" />
  </head>
  <body>
    <h1>HTML5 API 學習 PWA</h1>
    <p>用 Chrome / Edge 開啟此頁，網址列右側會出現「安裝」按鈕。</p>
    <p>iOS Safari：點擊「分享」→「加到主畫面」。</p>
  </body>
</html>
```

`manifest.webmanifest`：

```
{
  "name": "HTML5 API 學習 PWA",
  "short_name": "HTML5 API",
  "start_url": "./N01-pwa-basic.html",
  "scope": "./",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#3367d6",
  "icons": [
    { "src": "icons/icon.svg", "sizes": "any", "type": "image/svg+xml" }
  ]
}
```

欄位	用途
name	App 完整名稱（安裝對話框、啟動畫面顯示）
short_name	主畫面圖示下方的短名（建議 12 字以內，過長會被截斷）
start_url	從圖示開啟時的進入頁
scope	App 的導覽範圍；連到範圍外的網址會退回一般瀏覽器 UI（與 SW 的 scope 無關）
display	standalone 全螢幕無瀏覽器 UI / fullscreen / minimal-ui / browser
background_color	啟動畫面（splash screen）的底色
theme_color	標題列、狀態列底色
icons	各尺寸圖示陣列

檢驗工具：Chrome DevTools → Application → Manifest，會解析 manifest 並列出問題（缺欄位、icon 載不到等等）。

12.5 註冊 Service Worker

Service Worker 是背景執行的 JS 檔，獨立於頁面 — 頁面關了它還可以活著一段時間、處理推播、攔截網路請求。它的 lifecycle 分三階段：install → activate → 開始攔截 fetch。

範例：N02-pwa-service-worker.html（<head> 與 N01 相同，同樣要有 manifest 的設定）

```
<pre id="log"></pre>
<script>
  const log = document.querySelector("#log");
  const append = (msg) => (log.textContent += msg + "\n");

  if ("serviceWorker" in navigator) {
    navigator.serviceWorker
      .register("sw.js")
      .then((reg) => append(`SW 註冊成功, scope: ${reg.scope}`))
      .catch((err) => append(`SW 註冊失敗: ${err.message}`));
  } else {
    append("此瀏覽器不支援 Service Worker");
  }
</script>
```

sw.js（最小骨架，只記錄 lifecycle；範例專案沒有這個版本的檔案，專案裡的 sw.js 直接是 12.6 的完整版）：

```
self.addEventListener("install", () => {
  console.log("[SW] install");
});

self.addEventListener("activate", () => {
  console.log("[SW] activate");
});

self.addEventListener("fetch", (event) => {
```

```
console.log("[SW] fetch", event.request.url);
});
```

更新 SW：瀏覽器會比對 `sw.js` 的 byte 是否變動，有變才重新 install。改 `sw.js` 後在 DevTools → Application → Service Workers 勾「Update on reload」可強制重新註冊；或點「Unregister」全清掉重來。

scope：`sw.js` 放哪決定它能控制的路徑範圍。放在根目錄則控制整個網站；要更小可加 `register("sw.js", { scope: "./pwa/" })`。

12.6 離線快取

把 SW 升級為「快取靜態資源」，讓使用者離線也能打開頁面。最簡單的策略是 **cache-first**：先看快取、沒有才打網路。

範例：`N03-pwa-offline.html` — 頁面多了一個連線狀態指示器，用 `navigator.onLine` 讀目前狀態、監聽 `online` / `offline` 事件即時更新：

```
const status = document.querySelector("#status");
const updateStatus = () => {
  status.textContent = navigator.onLine ? "online" : "offline";
};
window.addEventListener("online", updateStatus);
window.addEventListener("offline", updateStatus);
updateStatus();
```

`navigator.onLine` 只代表「有沒有網路介面」，不保證真的連得到伺服器；要確認可用性還是得實際 `fetch` 看看。

Service Worker 的部分：

```
// sw.js (獨立 PWA 專案的版本；範例專案的差異見下方說明)
const CACHE = "html5-pwa-v1";
const ASSETS = [
  "./N01-pwa-basic.html",
  "./N02-pwa-service-worker.html",
  "./N03-pwa-offline.html",
  "./N04-pwa-install.html",
  "./manifest.webmanifest",
  "./icons/icon.svg",
];

self.addEventListener("install", (event) => {
  // 預先把資源放進快取
  event.waitUntil(caches.open(CACHE).then((c) => c.addAll(ASSETS)));
});
```

```

self.addEventListener("activate", (event) => {
  // 清掉舊版快取
  event.waitUntil(
    caches.keys().then((keys) =>
      Promise.all(keys.filter((k) => k !== CACHE).map((k) => caches.delete(k)))
    )
  );
});

self.addEventListener("fetch", (event) => {
  // cache-first: 先看快取，沒有才打網路
  event.respondWith(
    caches.match(event.request).then((cached) => cached || fetch(event.request))
  );
});

```

驗證：DevTools → Network → 切「Offline」，重新整理 N03 頁面仍能載入。

快取版本控管：版本字串（v1）變更時 activate 階段把舊版整個刪掉，避免不同版本資源混用。

不要快取所有東西：實務上要區分「靜態資源（cache-first）」與「API 呼叫（network-first 或 stale-while-revalidate）」，不同策略對應不同需求。

範例專案的差異：範例專案所有單元共用同一個 public/，且 SW 的 scope 是整個網站，因此 sw.js 的 fetch handler 另外加了條件，只攔截 PWA 相關資源（N0x 開頭等），避免 Service Worker 接管到其他單元的頁面。自己獨立的 PWA 專案就不需要這層判斷。

12.7 自訂安裝按鈕

Chrome / Edge 會在「符合資格」的 PWA 站點觸發 beforeinstallprompt 事件，這個事件可以攔下來、用自己的 UI 觸發安裝對話框，比預設的網址列圖示更顯眼。

範例：N04-pwa-install.html

```

<button id="installBtn" hidden>安裝到主畫面</button>
<p id="status">等待 beforeinstallprompt 事件...</p>

<script>
  const status = document.querySelector("#status");
  let deferredPrompt;

  window.addEventListener("beforeinstallprompt", (e) => {
    e.preventDefault(); // 攔下預設網址列圖示
    deferredPrompt = e;
    installBtn.hidden = false; // 顯示自己的按鈕
    status.textContent = "可安裝，按下方按鈕";
  });

```

```

installBtn.addEventListener("click", async () => {
  if (!deferredPrompt) return;
  deferredPrompt.prompt(); // 觸發瀏覽器安裝對話框
  const { outcome } = await deferredPrompt.userChoice;
  status.textContent = `使用者選擇: ${outcome}`; // accepted / dismissed
  deferredPrompt = null;
  installBtn.hidden = true;
});

window.addEventListener("appinstalled", () => {
  status.textContent = "已安裝!";
});

// 別忘了註冊 SW (beforeinstallprompt 的合規條件之一)
if ("serviceWorker" in navigator) navigator.serviceWorker.register("sw.js");

// 已經用安裝模式開啟時 (從主畫面圖示啟動) 顯示提示
if (window.matchMedia("(display-mode: standalone)").matches) {
  status.textContent = "已用安裝模式開啟 (standalone) ";
}
</script>

```

beforeinstallprompt 觸發的條件 (Chrome 的合規檢查) :

- 站點以 HTTPS 提供 (或 localhost)
- 有合法 manifest (含 name、start_url、display、icons 192/512 或 SVG)
- 有註冊 Service Worker (早期還要求 SW 有 fetch handler, Chrome 已逐步放寬)
- 同一網域沒被使用者拒絕安裝太多次

這份清單 Chrome 近年調整過多次, 以 <https://developer.chrome.com/docs/webapps/installability> 的現行版本為準。

iOS Safari 沒有 **beforeinstallprompt** — iOS 上要安裝必須引導使用者「分享 → 加到主畫面」, 自訂按鈕在 iOS 只能跳出說明文字。範例最後的

`matchMedia("(display-mode: standalone)")` 在 iOS 一樣可用來判斷是否已從主畫面開啟。

附錄一：Geolocation

`navigator.geolocation` 讓網頁取得使用者目前的地理位置。屬於隱私權資料, 瀏覽器會跳出權限對話框由使用者授權; 同時要求頁面以 HTTPS 提供 (localhost 例外), http 站點呼叫會直接失敗。

精準度依資料來源而定: 手機/桌機若有 GPS 最準 (誤差幾公尺); 其次是 WiFi 訊號定位 (幾十公尺); 最差只能用 IP 反查 (幾公里)。

A.1 取得目前座標

`getCurrentPosition(success, error, options)` 一次性取得位置；長時間追蹤要用 `watchPosition()`，回傳 `id` 可用 `clearWatch(id)` 停止。

範例：G01-geolocation.html

```
<pre id="info"></pre>
<script>
  navigator.geolocation.getCurrentPosition(
    (position) => {
      const lat = position.coords.latitude;
      const lng = position.coords.longitude;
      info.innerHTML = JSON.stringify({ lat, lng }, null, 4);
    },
    (error) => {
      // GeolocationPositionError
      // code: 1=PERMISSION_DENIED 2=POSITION_UNAVAILABLE 3=TIMEOUT
      console.log(error.code, error.message);
    }
  );
</script>
```

`position.coords` 還包含 `accuracy`（公尺）、`altitude`、`heading`、`speed`，視設備支援而定，不支援的欄位會是 `null`。

可選的 `options` 物件：

屬性	說明
<code>enableHighAccuracy</code>	要求高精度（會吃 GPS、較耗電）
<code>timeout</code>	等待毫秒數，超時觸發 <code>error</code> 且 <code>code === 3</code>
<code>maximumAge</code>	接受多舊的快取結果（毫秒）， <code>0</code> 表示強制重新取

A.2 座標搭配地圖

把座標標在地圖上是最常見的應用。下面用 Google Maps，需要先到 [Google Cloud Console](#) 申請 API key，把 `<script src>` 的 `key=` 後填上。

範例：G02-geolocation-map.html

```
<div id="myMap" style="width: 600px; height: 400px"></div>
<script src="https://maps.googleapis.com/maps/api/js?key=YOUR_KEY"></script>
<script>
  navigator.geolocation.getCurrentPosition(
    (position) => {
      const latlng = new google.maps.LatLng(
        position.coords.latitude,
        position.coords.longitude
      );

      // 建立地圖
      const map = new google.maps.Map(document.querySelector("#myMap"), {
```

```

        zoom: 13,
        center: latlng,
        mapTypeId: google.maps.MapTypeId.ROADMAP,
    });

    // 標記目前位置
    new google.maps.Marker({
        position: latlng,
        title: "你在這裡",
    }).setMap(map);
},
(error) => console.log(error.code, error.message)
);
</script>

```

`google.maps.Marker` 自 2024 年起被 Google 標為 deprecated，目前仍可用，新專案建議改用 `AdvancedMarkerElement`（需在 Map 選項加 `mapId`）。同步載入 `<script src="...maps/api/js">` 的方式也已非官方推薦寫法，正式專案請參考 Google 的 dynamic library import。

替代方案：OpenStreetMap + Leaflet — 不必 API key、免費，社群維護的開源圖資，也沒有上述 deprecation 問題。範例專案有一系列 Leaflet 範例，另有獨立講義《Leaflet 學習入門》（`leaflet-training.md`）逐一說明，本講義不重複。列表供參考：

檔案	內容
<code>L010-basic.html</code>	基本地圖設定
<code>L020-labels.html</code>	地圖標示
<code>L030-current-position.html</code>	搭配 <code>getCurrentPosition</code> 標出目前位置
<code>L040-watch-position.html</code>	用 <code>watchPosition</code> 持續追蹤位置
<code>L050-tile-layers.html</code>	切換地圖圖層
<code>L060-name-search.html</code>	地名搜尋 (Geocoding)
<code>L070-route-planning.html</code>	路線規劃 (Routing)
<code>L080-reverse.html</code>	反向地址查詢 (Reverse Geocoding)
<code>L090-search-cafe.html</code>	區域搜尋 (Overpass API)
<code>L100-markers.html</code>	從 JSON 載入多個標記 (<code>data/cafes.json</code>)
<code>L110-menu-to-loc.html</code>	連動選單與地圖

附錄二：用 OpenAI SDK 串接本機 Ollama

要在課堂示範 AI 串流回應，最方便的做法是裝 [Ollama](#) 在本機跑開源模型。Ollama 直接提供「OpenAI 相容 API」，所以前端用任何 OpenAI SDK（或直接打 REST）都能用。將來要切換成正版 OpenAI 只需改 `baseUrl` 與 `apiKey`，程式碼結構不必動。

B.1 環境準備

1. 到 <https://ollama.com/> 下載安裝 Ollama
2. 拉一個模型（這裡用 Google 的 gemma3）：

```
ollama pull gemma3
```

3. 啟動 Ollama 伺服器（安裝後背景常駐，預設 port `11434`）
4. 在 Express 專案安裝 OpenAI SDK：

```
npm i openai
```

Ollama 提供的 OpenAI 相容端點：`http://localhost:11434/v1/chat/completions`、`/v1/embeddings` 等等。`apiKey` 不檢查，給任何字串都可以。

文件：<https://docs.ollama.com/openai>

B.2 兩種架構

下面兩個範例對應兩種常見部署方式：

方式	範例	適用情境
前端直接呼叫 AI	J02-Ollama-chat-1.html	本機 Ollama demo（自己機器、無金鑰外洩疑慮）
透過後端中繼	J03-Ollama-chat-2.html	用真正的 OpenAI 必走的方式（保護 API key）

兩個範例共用一個 SSE 解析 helper `readSSE()`，正確處理 buffer 與事件邊界——串流可能在任意位置切割，不能假設一個 chunk 就是一個完整事件。它是針對 OpenAI 相容串流的簡化版：只處理 `data:` 行，遇到 `id:`、`event:` 或多行 `data:` 都不會處理。例如拿它去解 §8 的 `/try-sse`（每則訊息前有一行 `id:`）就會靜默失效。真正通用的解析器要逐行拆欄位。

B.3 前端直接呼叫 Ollama

最少結構：用 `fetch` 打 `http://localhost:11434/v1/chat/completions`，啟用 `stream: true` 後伺服器回 `text/event-stream` 格式。瀏覽器 `fetch` 不會自動解析 SSE，要自己累加緩衝、按 `\n\n` 切分事件。

範例：`J02-Ollama-chat-1.html`

```

<input type="text" id="chat" value="iphone 手機有什麼特色?" />
<button onclick="startAsk()">開始</button>
<div id="result_div" style="white-space: pre-wrap"></div>

<script>
  const AI_ROLE = "您是一個出色的 3C 產品介紹員，會使用正體中文回答";

  const startAsk = async () => {
    result_div.innerText = "";

    const response = await fetch("http://localhost:11434/v1/chat/completions", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        model: "gemma3",
        messages: [
          { role: "system", content: AI_ROLE },
          { role: "user", content: chat.value },
        ],
        stream: true,
      }),
    });

    if (!response.ok) {
      result_div.innerText = "回應發生錯誤";
      return;
    }

    await readSSE(response, (event) => {
      if (event === "[DONE]") return;
      const obj = JSON.parse(event); // Ollama 回的是完整 OpenAI 事件
      const text = obj.choices[0]?.delta?.content;
      if (text) result_div.innerText += text;
    });
  };

  // 簡化版 SSE 解析器，只處理 data: 行 (給 fetch 用; EventSource 是另一套)
  const readSSE = async (response, onEvent) => {
    const reader = response.body.getReader();
    const decoder = new TextDecoder("utf8");
    let buffer = "";
    while (true) {
      const { value, done } = await reader.read();
      if (done) break;
      buffer += decoder.decode(value, { stream: true });
      const events = buffer.split("\n\n");
      buffer = events.pop(); // 最後一段可能不完整，留下次再拼
      for (const ev of events) {
        if (ev.startsWith("data: ")) onEvent(ev.slice(6));
      }
    }
  }
}

```

```
};  
</script>
```

CORS：直接從瀏覽器打 Ollama，某些版本需要設環境變數 `OLLAMA_ORIGINS=*` 才允許跨來源。改用後端中繼可避開。

與附錄三 tunnel 互斥：若頁面是透過 `https://xxx.trycloudflare.com` 開啟，這裡的 `fetch("http://localhost:11434/...")` 會被瀏覽器以 mixed content 擋掉，手機更連不到你電腦的 localhost。用 tunnel 測試時只能走 B.4 的後端中繼版本。

B.4 透過後端中繼（保護 API key）

正式環境用 OpenAI 時，**API key 不能進前端**（很容易被偷）。要透過後端代為呼叫，再把回應串流轉發到前端。

後端：用 OpenAI SDK 連 Ollama

```
// index.js  
import { OpenAI } from "openai";  
  
// Ollama 提供 OpenAI 相容 API；apiKey 不檢查，給任何字串都可以  
const ollama = new OpenAI({  
  baseURL: "http://localhost:11434/v1/",  
  apiKey: "ollama",  
});  
  
app.post("/chat", async (req, res) => {  
  const message = req.body?.message;  
  if (!message) return res.status(400).json({ error: "沒有 message" });  
  
  res.writeHead(200, {  
    "Content-Type": "text/event-stream",  
    "Cache-Control": "no-cache",  
    Connection: "keep-alive",  
  });  
  
  const stream = await ollama.chat.completions.create({  
    model: "gemma3",  
    messages: [  
      {  
        role: "system",  
        content: "你是一個出色的 3C 產品介紹員，會使用正體中文、貼心精簡地回答",  
      },  
      { role: "user", content: message },  
    ],  
    stream: true,  
  });  
  
  // 用戶端關閉連線時中止 OpenAI 串流，避免繼續燒運算
```

```

req.on("close", () => stream.controller.abort());

for await (const event of stream) {
  const delta = event.choices[0]?.delta?.content;
  // JSON.stringify 處理可能含換行的字串 (SSE data 不能有原始換行)
  if (delta) res.write(`data: ${JSON.stringify(delta)}\n\n`);
}
res.write("data: [DONE]\n\n");
res.end();
});

```

切換成正版 OpenAI 只要改三處：

```

baseUrl: "https://api.openai.com/v1/",
apiKey: process.env.OPENAI_API_KEY,
model: "gpt-4o-mini", // 或其他 OpenAI 模型

```

前端：渲染 Markdown

讓 AI 回應可以正確顯示列表、程式碼區塊、粗體等格式，用 [marked.js](#) 把 Markdown 轉成 HTML。

範例：J03-011ama-chat-2.html

```

<input type="text" id="chat" value="iphone 手機有什麼特色?" />
<button onclick="startAsk()">開始</button>
<div id="result_div"></div>

<script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
<script>
  const startAsk = async () => {
    let content = "";
    result_div.innerHTML = "";

    const response = await fetch("/chat", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ message: chat.value }),
    });

    if (!response.ok) {
      result_div.innerText = "回應發生錯誤";
      return;
    }

    await readSSE(response, (event) => {
      if (event === "[DONE]") return;
      const text = JSON.parse(event); // 後端送的是 JSON.stringify 後的純文字
      content += text;
    });
  };

```

```
    result_div.innerHTML = marked.parse(content); // 邊收邊渲染
  });
};

// 同 J02-1 的 readSSE helper (省略)
</script>
```

為什麼不用 EventSource： EventSource 只支援 GET，這裡要 POST 訊息進來，所以用 fetch + 自己解 SSE。要做純訂閱情境 (GET only) 可以直接用 EventSource，見 §8 SSE。

每收一段就 marked.parse 一次會稍微浪費，但對短文字無感。要更省可以用 requestAnimationFrame 節流，或維護兩段（已完成的 HTML + 進行中的純文字）只重渲後者。

XSS： marked 從 v5 起不再內建消毒，模型輸出的任何 HTML 都會原樣塞進 innerHTML。正式環境務必搭配 DOMPurify：

```
result_div.innerHTML = DOMPurify.sanitize(marked.parse(content))。
```

附錄三：用 Cloudflare Tunnel 在本機開 HTTPS 測試環境

這份講義有好幾個 API 強制要 HTTPS 才能用：

- §12 PWA：Service Worker 註冊
- 附錄一 Geolocation：取得使用者位置
- Camera / Microphone (getUserMedia)、推播、加密儲存... (範例專案的 H01-webrtc-1.html、H02-webrtc-2.html 用 PeerJS 做一對一視訊通話，就需要這個)

localhost 算「安全來源 (secure context)」，自己機器測試沒問題；但要從手機或同學的電腦連進來測，就需要一個外部可達的 HTTPS 網址。自簽憑證可行但麻煩、要在每台裝置匯入信任。

Cloudflare Tunnel 提供 quick tunnel：免帳號、免費、一行指令，馬上得到

https://<隨機名>.trycloudflare.com 網址，把流量導到你 localhost 上的 server。

C.1 安裝 cloudflared

macOS

```
brew install cloudflared
```

或從 [GitHub Releases](#) 下載 cloudflared-darwin-arm64.tgz / -amd64.tgz，解壓後把 binary 放到 PATH (例如 /usr/local/bin/)。

Windows

```
winget install --id Cloudflare.cloudflared
```

或從 [GitHub Releases](#) 下載 `cloudflared-windows-amd64.msi` 雙擊安裝。

驗證

```
cloudflared --version
# cloudflared version 2026.x.x ...
```

C.2 啟動 quick tunnel

範例專案 Express 預設跑在 port `3031`。先確認 server 跑著：

```
npm run dev
# 伺服器啟動於通訊埠：http://localhost:3031
```

開另一個終端，把 `3031` 的 HTTP 流量映射成公開 HTTPS：

```
cloudflared tunnel --url http://localhost:3031
```

幾秒後會看到：

```
+-----+
| Your quick Tunnel has been created! Visit it at (it may take |
| some time to be reachable):                                |
| https://random-words-1234.trycloudflare.com                  |
+-----+
```

把這個網址貼到手機瀏覽器、給同事、或當成 webhook 接收端 — 連到的就是你 `localhost` 上的 server，自動 HTTPS。

WebSocket / SSE 都自動支援，不需要任何額外設定 — 對 §8 SSE 與 §9 WebSocket 範例都能直接用。

C.3 典型應用情境

情境	為什麼需要 tunnel
手機測試 PWA (§12)	Service Worker、Push 等都需要 HTTPS
手機測 Geolocation (附錄一)	真正的 GPS 在手機；瀏覽器要 HTTPS 才回座標
跟同事示範	不必部署，把網址貼過去就能看
Webhook 接收 (LINE/Stripe...)	第三方服務的 callback 必須打到外部可達的 HTTPS 端點

C.4 注意事項與進階用法

- **每次重啟 URL 會變**：quick tunnel 是隨機產生的，停掉再開一次就是新網址。要把網址做成 QR code 給學生掃，記得每堂課重產。

- **不適合正式環境**：免費的 quick tunnel 有流量上限與限速，且 Cloudflare 不對可用性與效能做任何書面保證（沒 SLA — Service Level Agreement，服務水準協定）。突然斷線、被限速都沒得申訴。正式服務該用付費方案或自架。
- **想固定網域**（例如 `dev.myapp.com`）：需要 Cloudflare 帳號 + 你的網域托管在 Cloudflare DNS：

```
cloudflared login # 開瀏覽器授權
cloudflared tunnel create my-tunnel
cloudflared tunnel route dns my-tunnel dev.myapp.com
cloudflared tunnel --url http://localhost:3031 run my-tunnel
```

`--url` 是 `tunnel` 層級的旗標，要放在 `run` 之前。具名 tunnel 的常規做法是改在 `~/.cloudflared/config.yml` 用 `ingress` 設定對應，之後只要 `cloudflared tunnel run my-tunnel`。

- **看流量**：`cloudflared tunnel --url http://localhost:3031 --loglevel debug`，會把每個請求 log 出來。
- **同源限制與 CORS**：tunnel 對外是個全新網域（`*.trycloudflare.com`），跟你 `localhost` 不同源。對前端來說沒影響（瀏覽器只看到 tunnel 網域）；但若你的 server 有依賴 `Origin` 標頭做白名單，記得加進去。
- **mixed content**：頁面走 HTTPS 之後，頁面內任何 `http://localhost:...` 的 `fetch` 都會被瀏覽器封鎖。附錄二 B.3 「前端直連 Ollama」在 tunnel 下不能用，要改走 B.4 的後端中繼。